

海洋结构分析通用软件 SAM

接口说明文档

V1.0

20260605



开发单位：中国船舶科学研究中心及下属中船奥蓝托无锡软件技术有限公司

地址：江苏省无锡市滨湖区山水东路 222 号

网址：www.cssrc.com.cn

目录

1 简介	1
2 Python 二次开发	2
2.1 二次开发简介	2
2.2 脚本二次开发	2
2.2.1 脚本编写	2
2.2.2 脚本执行	3
2.2.3 实例 1: 参数化创建加筋板前处理建模	13
2.2.4 实例 2: 参数化加筋板计算及后处理	17
2.3 界面二次开发	19
2.3.1 启动项	19
2.3.2 通过脚本初始化需要加载的模块	20
2.3.3 模块	21
3 接口说明	29
3.1 Session 内置方法	29
3.1.1 Viewport()	29
3.1.2 Viewport.makeCurrent()	30
3.1.3 Viewport.maximize()	30
3.1.4 Viewport.partDisplay.setValues()	30
3.1.5 Viewport.partDisplay.meshOptions.setValues()	31
3.1.6 Viewport.setValues()	32
3.1.7 Viewport.view.fitView()	32
3.1.8 Viewport.view.setValues()	33
3.1.9 Viewport.view.setProjection()	33
3.1.10 Viewport.assemblyDisplay.setValues()	34
3.1.11 Viewport.odtDisplay.setFrame()	35
3.1.12 Viewport.odtDisplay.display.setValues()	35
3.1.13 Viewport.odtDisplay.setPrimaryVariable()	36
3.2 mdb 内置方法	36
3.2.1 mdb.models	36
3.2.2 Part()	39
3.2.3 Material()	48
3.2.4 Section()	51
3.2.5 Assign()	53
3.2.6 Profile()	54
3.2.7 rootAssembly()	57
3.2.8 Step()	60
3.2.9 BoundaryCondition()	67
3.2.10 Load()	69
3.2.11 Interaction()	88
3.2.12 Mesh()	90
3.2.13 Job()	102
3.3 Odb 内置方法	103
3.3.1 importHDF5FileNewOdb()	103
3.3.2 Model.exportOdbToVtk()	104

1 简介

海洋结构分析通用软件(SAM), 以结构有限元分析为核心, 对标 Nastran、Ansys、Abaqus 等商业软件, 支持静力、模态、稳态、瞬态、非线性等常用分析, 精度和商软误差<0.1%, 效率和商软相当。具有自主化、高精度、专业化、全面性、开放性和可靠性等特色。

当前版本使用说明

- 当前版本有限元分析支持静力分析（包括线性，材料非线性，几何非线性），模态分析（包括结构和声学模态分析），稳态动力学分析（即谐响应分析，包括模态叠加法和直接法），响应谱分析，瞬态动力学分析。SAM 整体分析功能和商软 Nastran、Ansys、Abaqus 对比如下：

分析类型		商软				SAM
		重要度	MSC.Nastran	Ansys	Abaqus	
1. 静力分析	通用线性静力	5	√	√	√	√
	线性屈曲分析	2	√	√	√	×
	惯性释放	2	√	√	√	√
2. 模态分析	模态分析	5	√	√	√	√
3. 稳态响应分析	正弦响应分析	4	√	√	√	√
	响应谱分析	4	√	√	√	√
	随机响应分析	2	√	√	√	×
4. 瞬态动力学	隐式动力学	3	√	√	√	√
	显式动力学	4	×	×	√	×
5. 非线性分析	材料非线性	5	√	√	√	√
	几何非线性	3	√	√	√	√
	边界非线性	3	×	√	√	×

- 目前，SAM 有限元求解器的精度和商软 CAE 一致，SAM 包括 Abaqus 和 Nastran 单元，其中 SAM 采用 Abaqus 单元时所有模型线性/非线性精度和 Abaqus 的误差<0.1%, SAM 采用 Nastran 单元时所有模型线性精度和 Nastran 误差<1%。同时，由于各个有限元求解器的算法不同，不同软件之间的计算结果会有偏差，默认情况下，SAM 采用 Abaqus 单元，此时可能会和 Nastran

有差异。想要 SAM 和 Nastran 的分析结果更加接近，那么需要在 SAM 中改为 Nastran 单元。

- 当前版本只支持简单的几何体创建功能，对复杂模型，均采用导入 Abaqus 的 *.inp 或者 Nastran 的 bdf 的有限元模型的方式，如原有模型采用 Ansys 的 cdb 等模型，需要通过 Abaqus 转成 inp 后再导入到 SAM 中。
- 单位制：SAM 无单位制的限定，对所有单位制都可执行，只要各种物理量满足同一个单位制就行，譬如如果用 SI（mm），那么几何尺寸就是 mm，而杨氏模量用 MPa。

2 Python 二次开发

2.1 二次开发简介

软件提供了 Python 扩展接口，开发人员可进行命令行脚本和界面的二次开发。

2.2 脚本二次开发

软件采用 Python 进行脚本二次开发，可用于命令行脚本到创建定制的应用程序。

2.2.1 脚本编写

SAM 界面上的所有操作都记录在了 CommandHistory 窗口。

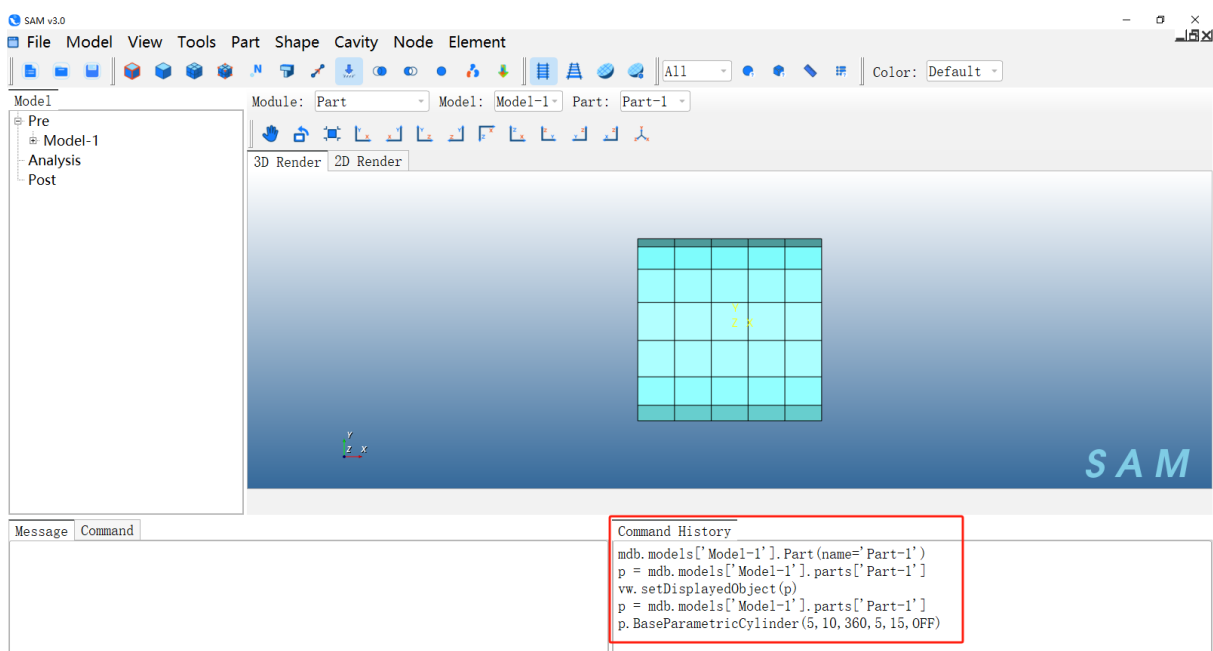


图 1 CommandHistory 窗口显示操作记录

在 **CommandHistory** 窗口中拷贝该脚本命令，然后新建.py 格式的文件，将脚本命令粘贴进去，即为一个自动化脚本文件。

✧ 脚本可根据实际需要在文本编辑器中修改

2.2.1.1 脚本段落：引用

脚本使用时按需引用 **SAM** 或第三方库的模块，如：

```
fromsamimport*
```

```
fromsamConstantsimport*
```

表示引用 **SAM** 的执行模块，其中定义了一些 **SAM** 常用变量和参数，在使用无界面执行脚本操作时需要引用。

2.2.1.2 脚本段落：命令语句

脚本使用时的命令语句，用来发送操作指令。

```
#定义模型变量
model=mdb.models['Model-1']
#创建Part
mdb.models['Model-1'].Part('Cylinder-1')
#定义部件变量
p = mdb.models['Model-1'].parts['Cylinder-1']
#参数化建模：圆柱壳
p.BaseParametricCylinder(5,10,360,5,40,OFF)
#定义装配体变量
assembly=model.rootAssembly
#创建实例
instance=assembly.Instance(name='Cylinder-1-1',part=p)
#创建静力分析步
model.StaticStep(name='Step-1',previous='Initial',timePeriod=1,initialInc=1,minInc=1E-005,maxInc=1)
#创建材料
material=model.Material(name='Material-1')
#材料密度
material.Density(table=((7800.0,)),)
#材料弹性属性
material.Elastic(table=((2.1e+11,0.3)),)
#创建壳单元截面
model.HomogeneousShellSection(name='Section-1',material='Material-1',thickness=0.001)
```

图 2 部分脚本命令

2.2.2 脚本执行

2.2.2.1 SAM 界面执行自动化脚本

打开 **SAM**。

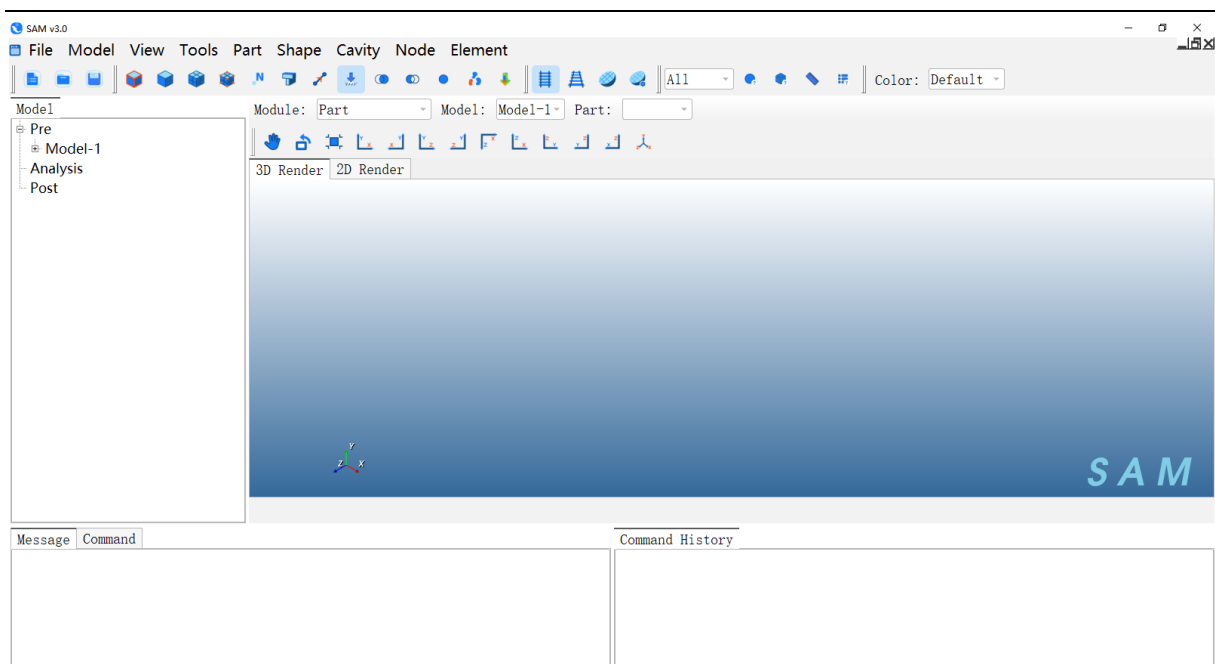


图 3 打开 SAM 界面

在 SAM 软件中点击菜单 **File→RunScript...**。

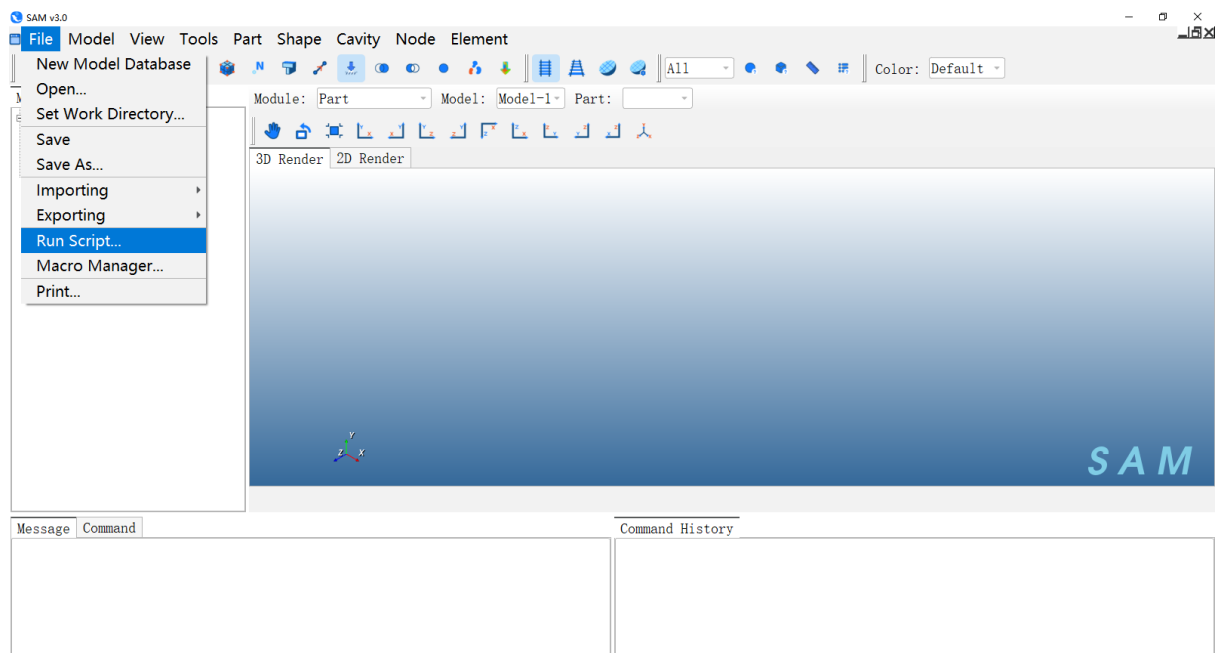


图 4 导入脚本命令

选择脚本文件，譬如安装目录下\Example\Scripts\StaticAnalysis.py

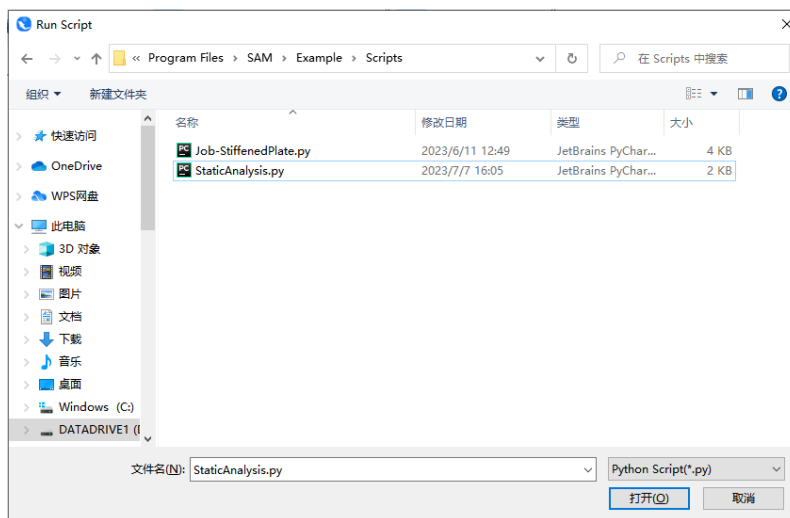


图 5 选择脚本命令文件

如下图所示，自动生成 python 脚本命令流实现模型自动创建

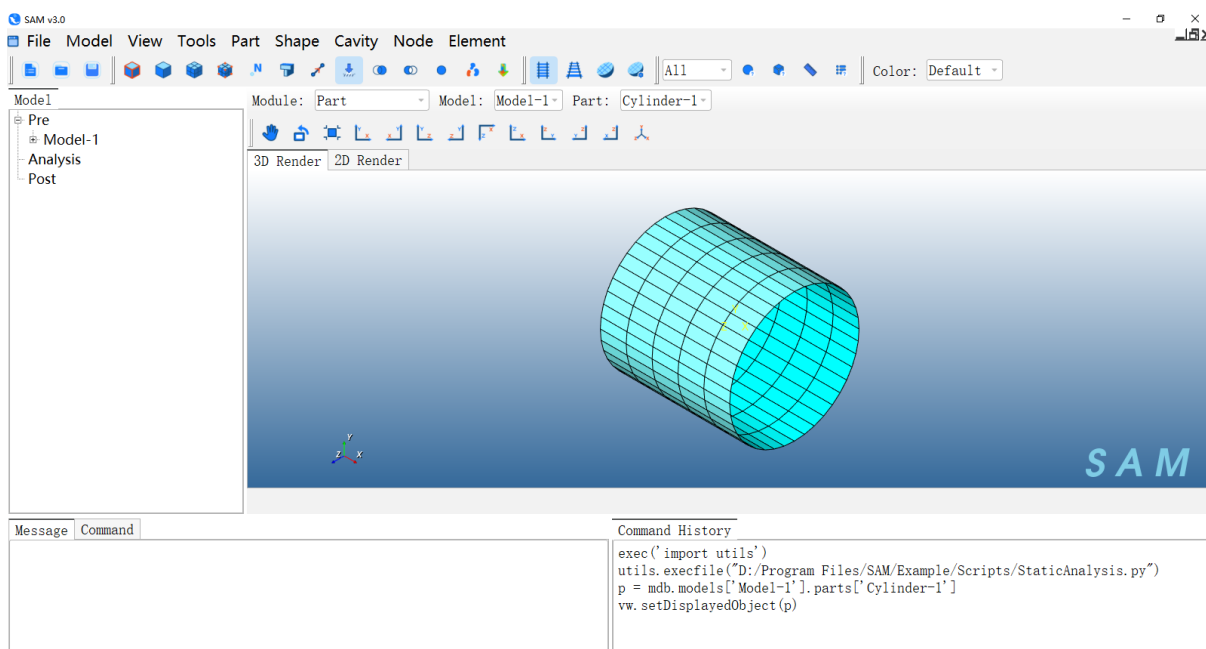


图 6 使用脚本命令实现模型自动创建

切换 Module 至 Job 模块，点击菜单 **Job→Manager**，可发现已经有一个 Job。

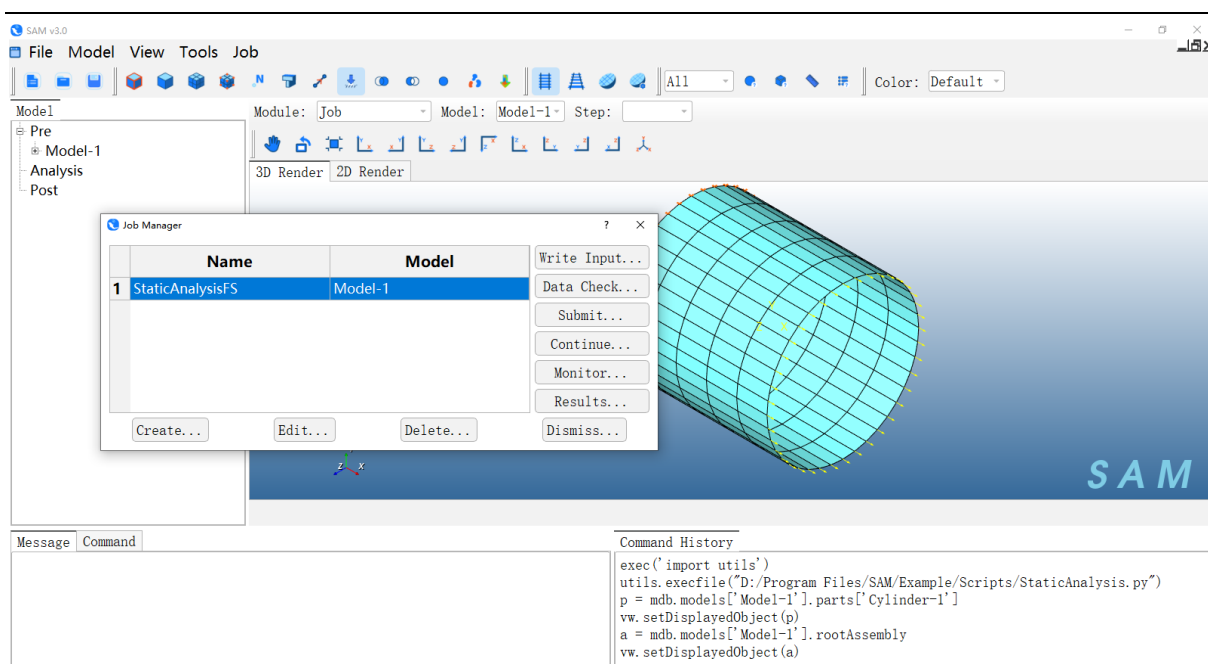


图 7 JobManager

可直接点击 **Submit** 进行提交任务计算及后处理分析等后续操作。

2.2.2.2 无界面执行自动化脚本

SAM 支持通过命令行不打开界面直接后台执行创建的自动化脚本。如下图所示，在系统的命令行窗口按规定格式输入语句，即可后台执行自动化脚本。

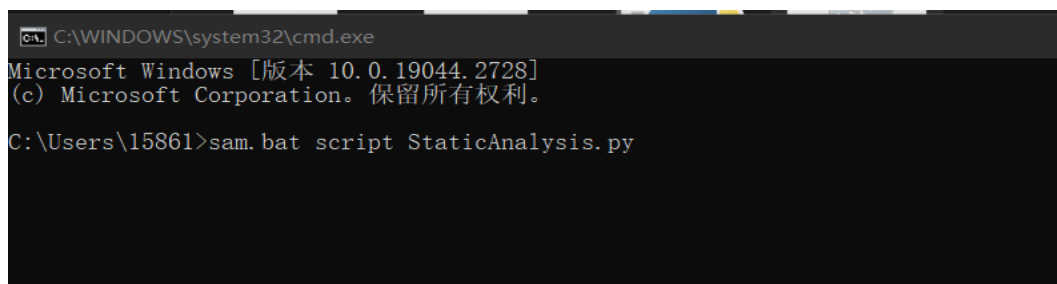


图 8 命令执行方式

命令语句格式为：**sam.batscript[script]**

其中，**sam.bat** 为固定项，表示软件启动的批处理文件；

script 为固定项，表示软件启动时为 **python** 模式；

[script] 为修改项，用来指定运行的脚本文件名称。

- ✧ 当需要运行的脚本文件不在当前目录时，需要输入脚本文件完整的路径。
- ✧ 后台运行的自动化脚本中不能包含对渲染模型的操作，比如 **setDisplayObject**

拷贝安装目录下 **\Example\Scripts\StaticAnalysis.py** 到某个单独目录（譬如 **D:\temp**）下

2.2.2.2.1 第一次运行脚本

1) 任意目录打开命令行，运行

`sam.batscriptStaticAnalysis.py`（脚本所在目录）

`sam.batscriptD:\\temp\\StaticAnalysis.py`（其他目录则需要指定脚本所在路径）

此时，SAM 会自动创建圆柱模型，在当前目录生成输出 `inp` 文件 `StaticAnalysisFS.inp`。

2) 可以用文本编辑器打开结果文件，里面记录了模型的节点、单元、材料属性、分析步、约束、载荷等信息。

3) 可以打开 SAM 查看模型信息，点击 **File→Importing→Model**

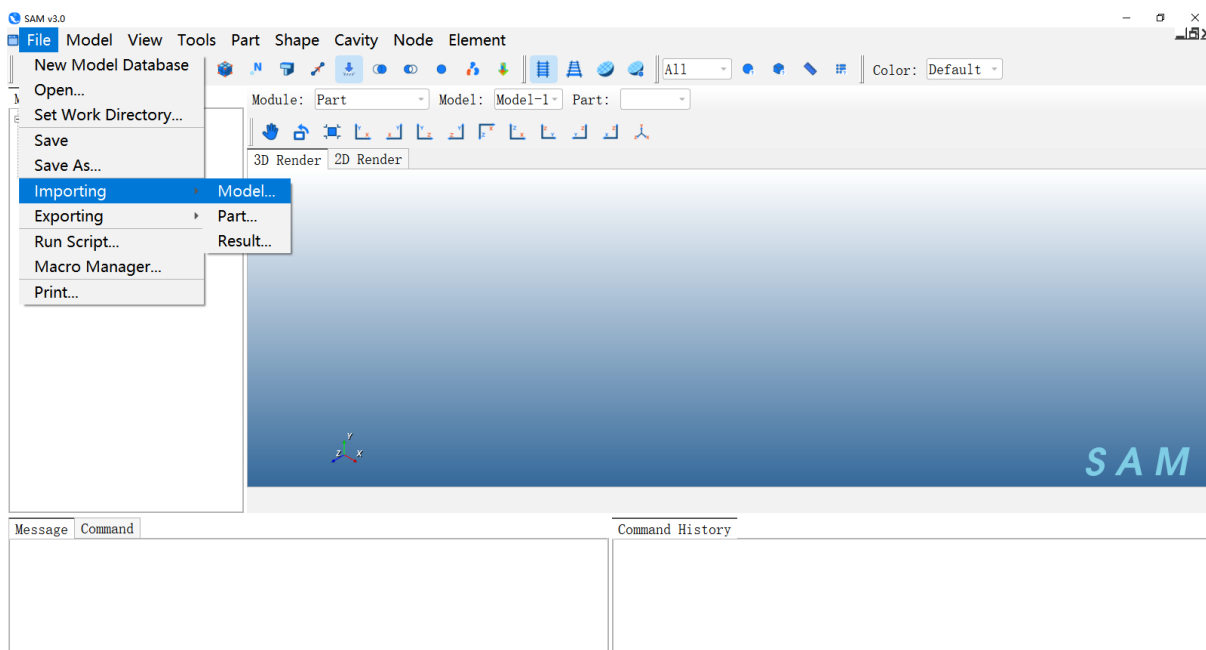


图 9SAM 打开文件

选取 `StaticAnalysisFS.inp` 结果文件

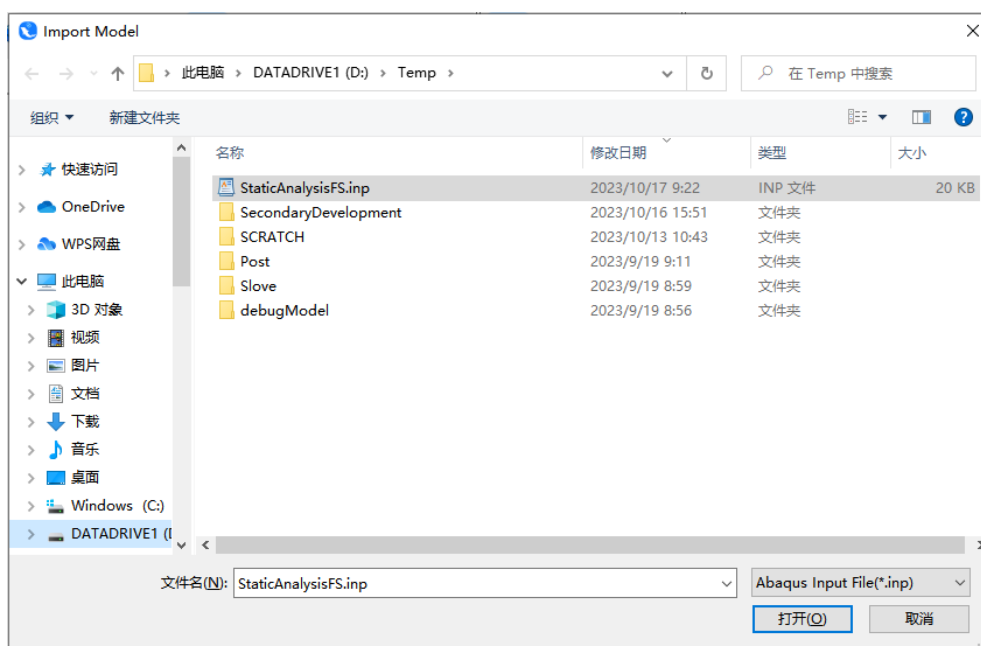


图 10SAM 导入.inp 文件

导入.inp 查看模型。

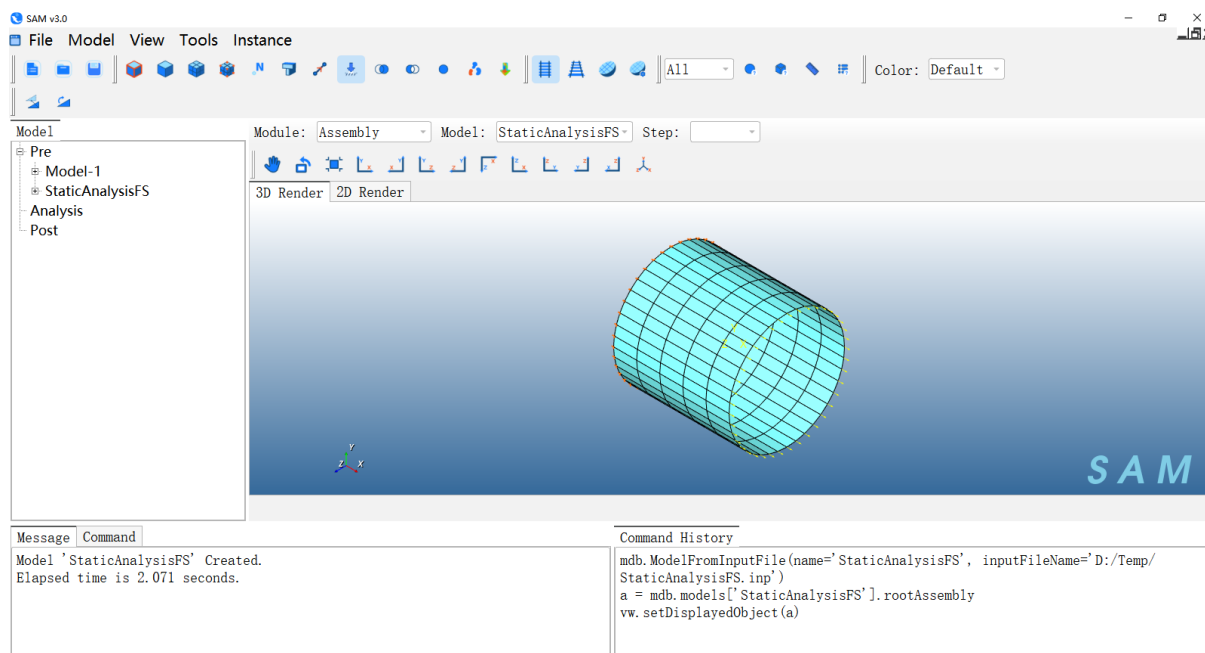


图 11 第一次运行脚本后的结果

2.2.2.2.2 第二次运行脚本

关闭 SAM，在文本编辑器中修改脚本 `StaticAnalysis.py`，比如把壳单元的厚度从 0.001 改成 0.002，

```

Job-singleSphere.py x StaticAnalysis.py x StaticAnalysis.py x
7 assembly=model.rootAssembly
8 instance=assembly.Instance(model.parts["Cylinder-1"])
9
10 #session.ballValue()
11
12 model.StaticStep("Step-1", "Initial")
13
14 # Create Material
15 material=model.createMaterial("Material-1")
16 material.Elastic([[float(2.1e11), float(0.3)],])
17 material.Density([[float(7800),],])
18
19 model.HomogeneousShellSection("Section-1", "Material-1", 1, 0.002)
20 mdb.models["Model-1"].parts["Cylinder-1"].createSet('Set-Section', 2, [i for i in range(165)])
21 mdb.models["Model-1"].parts["Cylinder-1"].SectionAssignment('Section-1', 'Set-Section')
22 # Create BC
23 assembly.createSetByUserWithInstance('Set-BC', 1, ['Cylinder-1-1'], [[0, 1, 2, 3, 11, 12, 13, 21, 22, 23, 31, 32, 33, 41,
24 setSelected=model.rootAssembly.sets["Set-BC"]
25 model.createBC("BC-1", "Step-1", setSelected, 1)
26 assembly.createSetByUserWithInstance('Set-Load', 1, ['Cylinder-1-1'], [[8, 9, 10, 18, 19, 20, 28, 29, 30, 38, 39, 40, 41,
27 setSelected2=model.rootAssembly.sets["Set-Load"]
28 model.createLoad("Load-1", "Step-1", setSelected2, 1, ["0", "1", "1"])
29
30 # Create Job

```

图 12 修改脚本

然后按上述方式再次运行该脚本，重新打开 SAM, 查看 .inp 文件。Module 切换到 Profile, 点击 Section 下的 Manager。

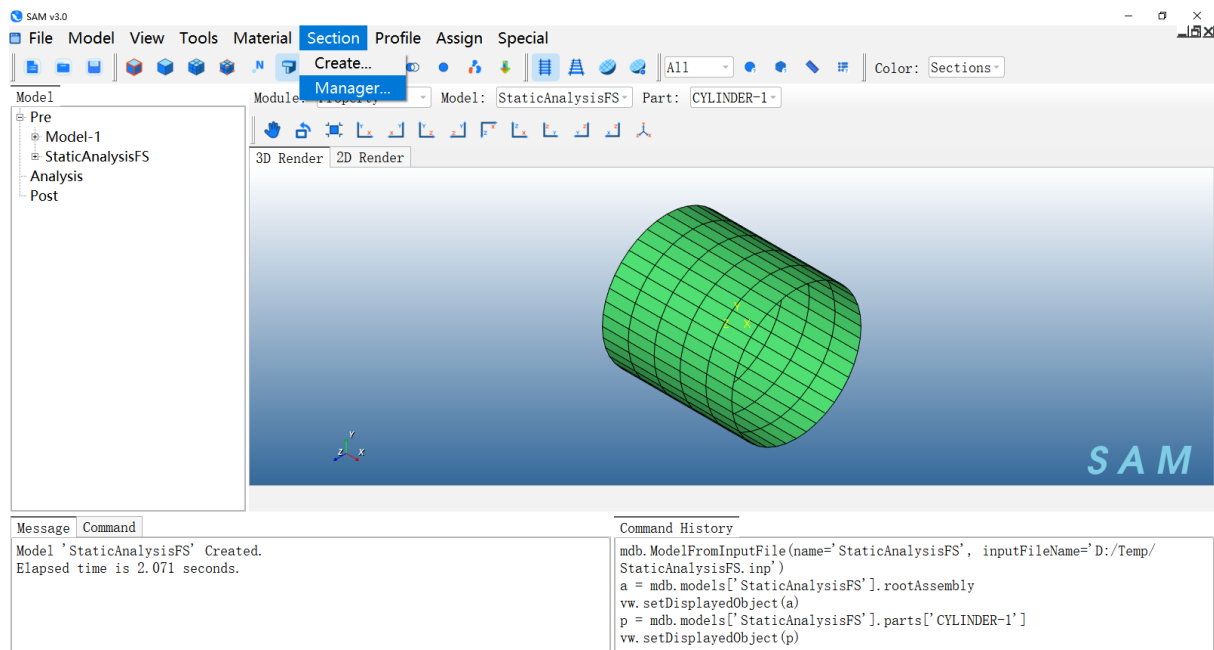


图 13 第二次运行脚本后的模型

在 Manager 窗口中选择 Section-1，点击 Edit 功能按钮，可以看到壳属性的厚度变成了 0.002

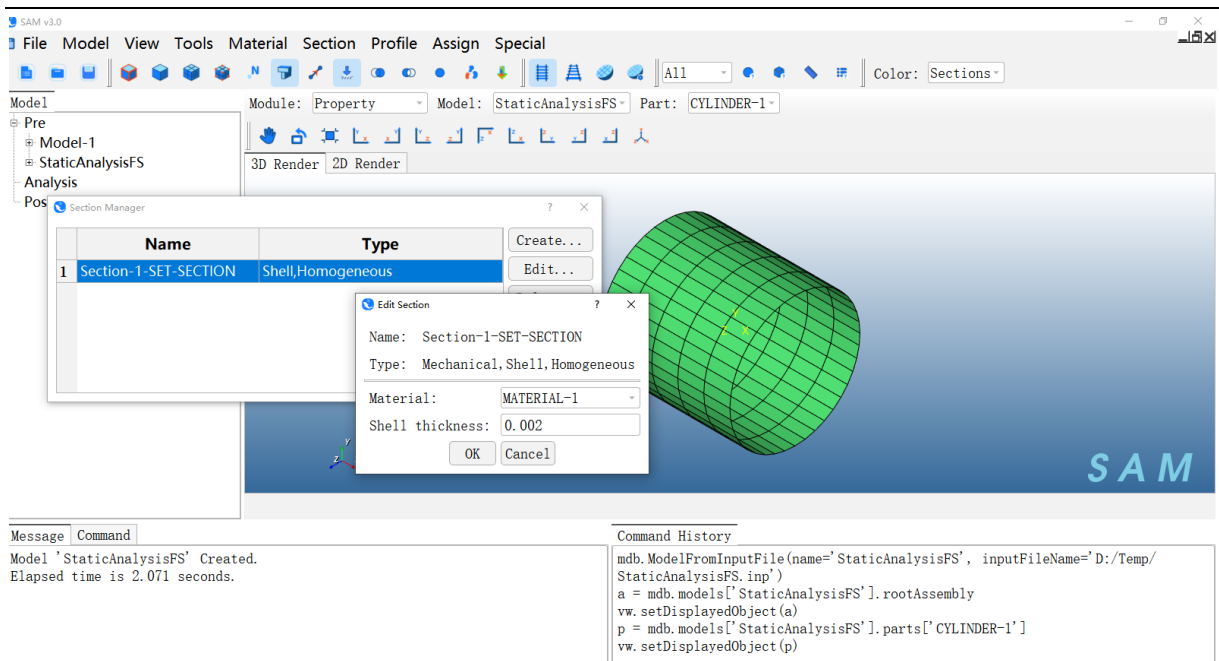


图 14 查看厚度数据

2.2.2.3 SAM 界面分步执行脚本命令

SAM 支持在界面上 **Command** 窗口通过命令进行建模及仿真计算。以创建材料为例：

打开 SAM，命令窗口位于左下方：

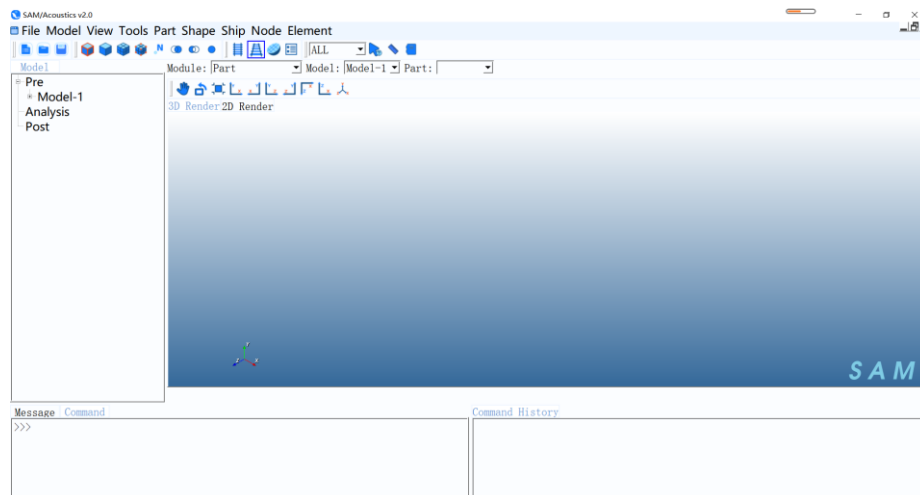


图 15 使用脚本命令实现模型创建以及网格划分

输入创建新材料的命令后，回车：

```

mdb.models['Model-1'].Material('MaterialByPython')

```

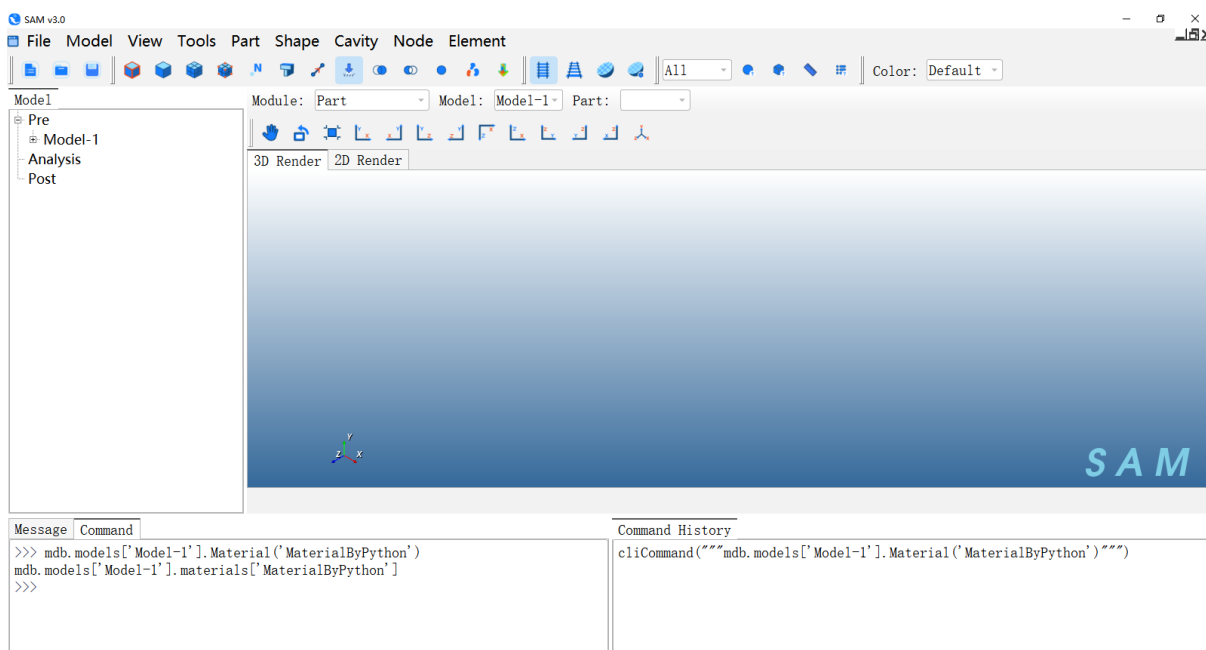


图 16 输入命令

点击 **Module**，切换到 **Property** 模块。点击菜单 **Material**→**MaterialManager**。

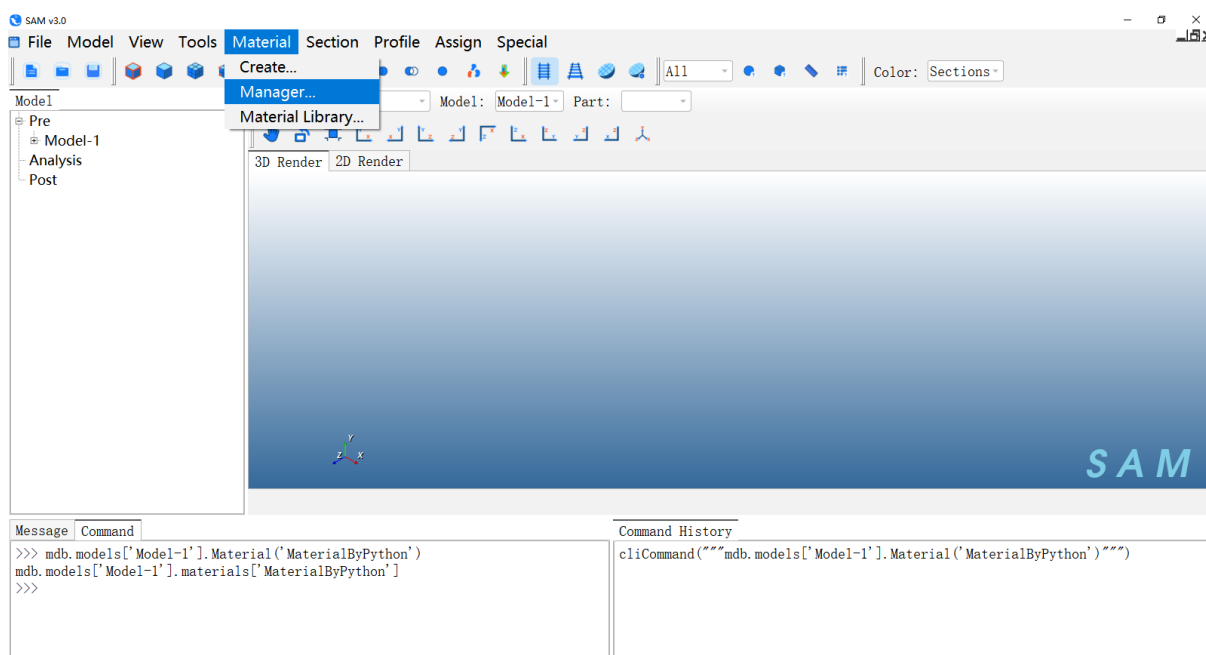


图 17 查看材料

在 Manager 窗口中能够查看到名为“MaterialByPython”的材料名。

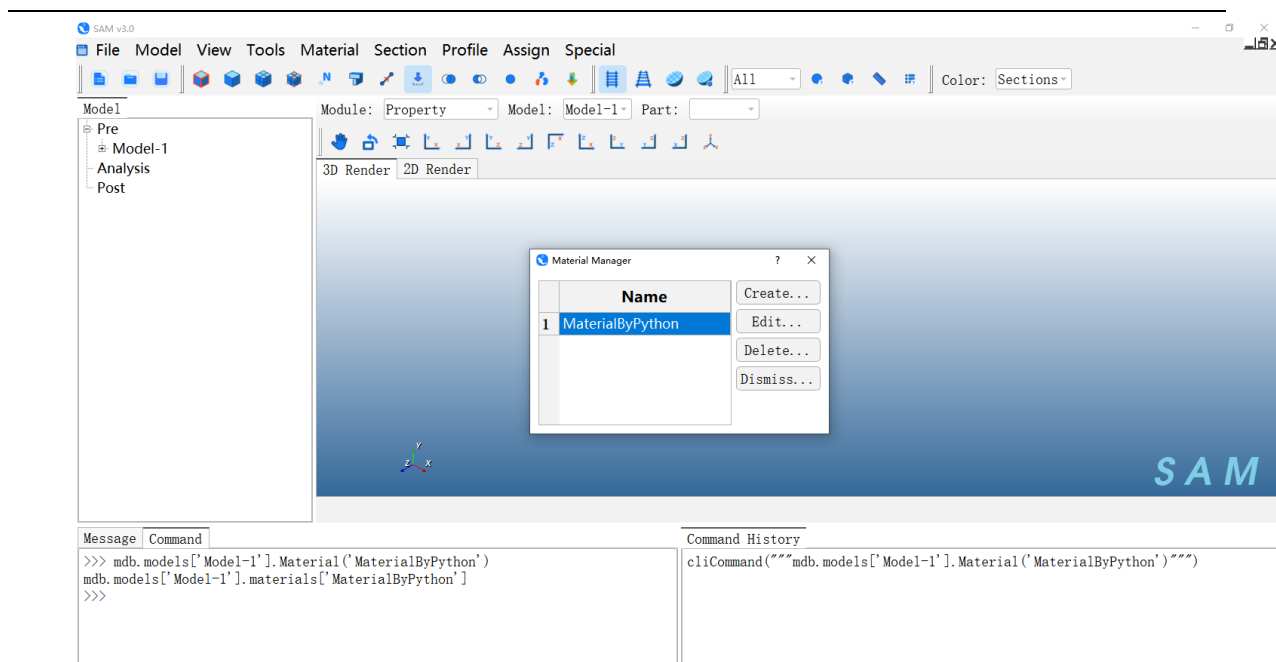


图 18 查看脚本创建的材料

点击 **Edit**，可发现材料参数为空。

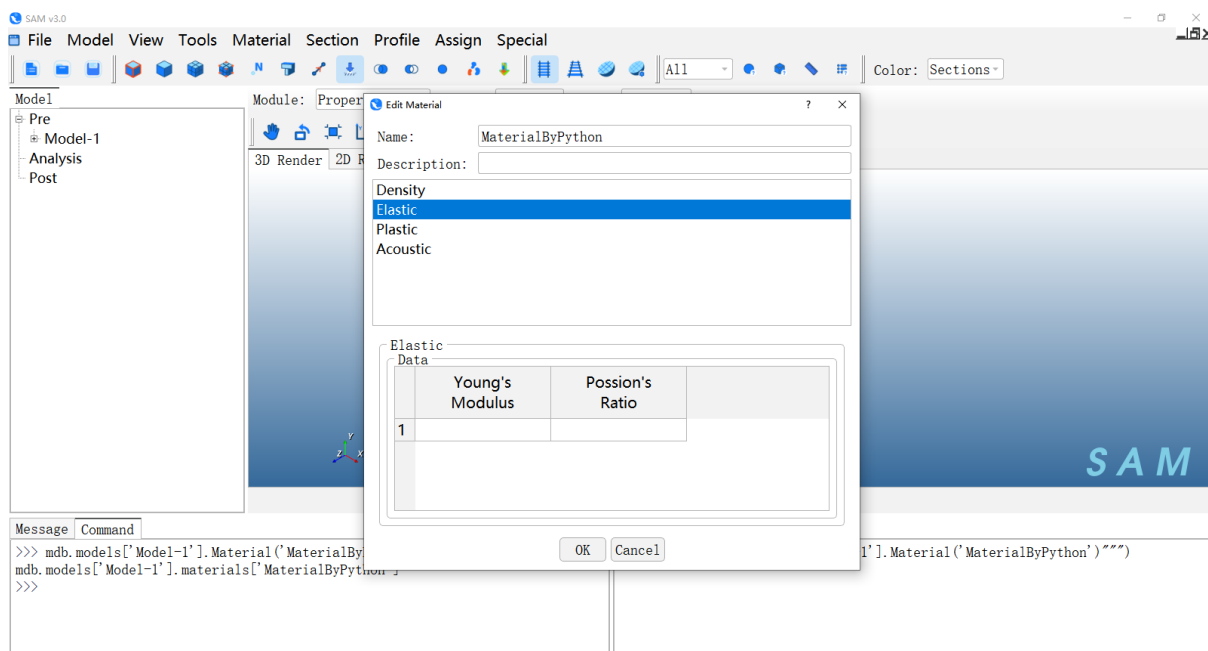


图 19 空数据材料

用户可尝试下面命令流，运行完毕继续通过上步 **Manager** 界面查看材料数据。

```
mdb.models['Model-1'].materials['MaterialByPython'].Density([[7800.0],])
```

```
mdb.models['Model-1'].materials['MaterialByPython'].Elastic([[210000000000.0,0.3]])
```

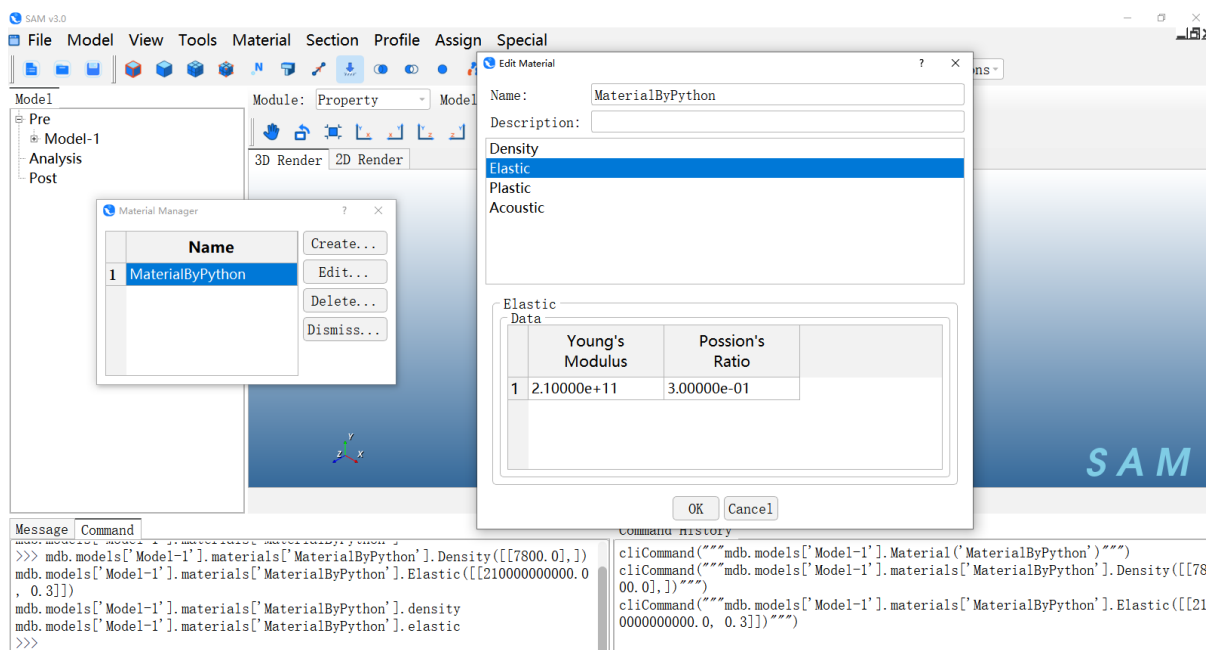



图 20 查看材料

2.2.3 实例 1：参数化创建加筋板前处理建模

2.2.3.1 实例说明

此实例为加筋板板架结构的前处理建模。

通过本实例用户将了解：

- SAM 软件中完成参数化的典型加筋板建模的脚本命令；
- SAM 中材料设置、分析步设置、边界条件和载荷加载脚本命令。

2.2.3.2 脚本命令及说明

#引用

fromsamimport*

fromsamConstantsimport*

#重置模型状态

Mdb()

#定义加筋板参数化建模所需参数

Dx=1

Nx=40

Dy=1

```
Ny=40
l=Dx*Nx
w=Dy*Ny
pressureMagnitude=1
#创建材料
mdb.models['Model-1'].Material(name='AISI1020')
#材料密度
mdb.models['Model-1'].materials['AISI1020'].Density(table=((7900.0,)),)
#材料杨氏模量和泊松比
mdb.models['Model-1'].materials['AISI1020'].Elastic(table=((200000000000.0,0.29)),)
#创建型材：T 型梁
mdb.models['Model-1'].TProfile(name='T_211-41',b=0.038,h=0.044,l=0.0,tf=0.005,tw=0.005)
mdb.models['Model-1'].TProfile(name='XC214-4',b=0.111,h=0.069,l=0.0,tf=0.008,tw=0.008)
#创建部件
mdb.models['Model-1'].Part(name='Part-1')
#部件变量
p=mdb.models['Model-1'].parts['Part-1']
#显示部件
vw.setDisplayedObject(p)
#构建加筋板参数化建模命令
GeomX=""
GeomY=""
#构建 X 方向上筋的个数及位置
for i in range(0,Nx+1):
    GeomX=GeomX+str(i*Dx)
    if i<Nx:
        GeomX=GeomX+", "
```

continue

#构建 Y 方向上筋的个数及位置

for i in range(0, Ny+1):

GeomY=GeomY+str(i*Dy)

if i<Ny:

GeomY=GeomY+", "

continue

#参数化构建加筋板

p.BaseParametricStiffenedPlate(str(GeomX),str(GeomY),"0,0,0","0,0,0","0,0,0",0,1,0)

#基于型材属性创建梁截面

mdb.models['Model-1'].BeamSection(name='Section-beam-x',integration=DURING_ANALYSIS,profile='T_211-41',material='AISI1020')

mdb.models['Model-1'].BeamSection(name='Section-beam-y',integration=DURING_ANALYSIS,profile='XC214-4',material='AISI1020')

#创建壳 Section，定义板厚

mdb.models['Model-1'].HomogeneousShellSection(name='Section-shell',material='AISI1020',thickness=0.1)

#获取指定 set 变量

regionX=mdb.models['Model-1'].parts['Part-1'].sets['Set-beam-x']

regionY=mdb.models['Model-1'].parts['Part-1'].sets['Set-beam-y']

#分配截面给指定 set 中包含的单元

p.SectionAssignment(region=regionX,sectionName='Section-beam-x',n1=(0,1,0))

p.SectionAssignment(region=regionY,sectionName='Section-beam-y',n1=(-1,0,0))

region=mdb.models['Model-1'].parts['Part-1'].sets['Set-shell']

mdb.models['Model-1'].parts['Part-1'].SectionAssignment(region=region,sectionName='Section-shell')

#装配体变量

```
a=mdb.models['Model-1'].rootAssembly
```

```
#显示装配体
```

```
vw.setDisplayedObject(a)
```

```
p=mdb.models['Model-1'].parts['Part-1']
```

```
#基于部件创建实例
```

```
a.Instance(name='Part-1-1',part=p)
```

```
#创建静力分析步
```

```
mdb.models['Model-1'].StaticStep(name='Step-1',nlgeom=OFF,timeIncrementationM  
ethod=AUTOMATIC,previous='Initial',timePeriod=1,initialInc=1,minInc=1E-005,maxIn  
c=1,maxNumInc=100)
```

```
#获取实例中所有节点的集合变量
```

```
n1=a.instances['Part-1-1'].nodes
```

```
#根据包围盒确定节点集合
```

```
nodes1=n1.getByBoundingBox(-1.e-6,-1.e-6,-1.e-6,1.e-6,w+1e-6,1.e-6)
```

```
#创建节点 set
```

```
a.Set(nodes=nodes1,name='Set-1')
```

```
#获取 set 变量
```

```
set1=mdb.models['Model-1'].rootAssembly.sets['Set-1']
```

```
#创建固支约束
```

```
mdb.models['Model-1'].EncastreBC(name='BC-1',createStepName='Step-1',region=s  
et1)
```

```
#装配体变量
```

```
a=mdb.models['Model-1'].rootAssembly
```

```
vw.setDisplayedObject(a)
```

```
#获取实例中所有单元的集合
```

```
f1=a.instances['Part-1-1'].elements
```

```
#创建 surface 表面
```

```
a.Surface(side1Elements=f1,name='Surface-1')
```

```
#获取表面区域
```

```
region=a-surfaces['Surface-1']
```

```

#创建均布压力载荷
mdb.models['Model-1'].Pressure(name='Load-Pressure',createStepName='Step-1',region=region,magnitude=pressureMagnitude)
#工作名称变量
jobName='Job-StiffenedPlate'
#创建工作任务
mdb.Job(name=jobName,model='Model-1',h5=ON)
#输出 inp 文件
mdb.jobs[jobName].writeInput()

```

2.2.4 实例 2：参数化加筋板计算及后处理

2.2.4.1 实例说明

此实例为加筋板板架结构的静力分析计算及后处理操作。

通过本实例用户将了解：

- SAM 中导入模型执行计算操作。
- SAM 中后处理操作。

2.2.4.2 脚本命令及说明

```

#导入模型
jobName='Job-StiffenedPlate'
inputFilePath='D:/Temp/Job-StiffenedPlate.inp'
mdb.ModelFromInputFile(name=jobName,inputFileName=inputFilePath)
#计算
mdb.jobs[jobName].submit()
#后处理结果名称变量
resultName=jobName+'.h5'
#加载后处理结果
session.importHDF5File(resultName)
#显示后处理结果
vw.setDisplayedObject(resultName)

```

```
#后处理数据变量

hdf5=session.odbs[resultName]

#获取分析步名称

stepName=list(hdf5.steps.keys())[0]

#根据名称获取分析步变量

hdf5Step=hdf5.steps[stepName]

#获取增量步 Index

frameIndex=len(hdf5Step.frames)-1

#获取增量步变量

frame=hdf5Step.frames[frameIndex]

#定义场量名称： 位移

fieldName_U='U'

#获取位移变量

field=frame.fieldOutputs[fieldName_U]

#定义分量名称： 幅值

componentName_U='Magnitude'

#刷新渲染窗口

vw.show()

#显示后处理结果

vw.setDisplayedObject(resultName)

#设置后处理显示的时间步与增量步

vw.odbDisplay.setFrame(str(stepName),frameIndex)

#设置后处理显示的场量与分量

vw.odbDisplay.setPrimaryVariable(str(fieldName_U),field.position,1,str(componentName_U))

#输出位移结果图片

vw.printPNGOffScreen("disp.png")

#定义场量名称： 应力

fieldName_S='S'

#获取应力变量
```

```

field1=frame.fieldOutputs[fieldName_S]
#定义分量名称：米塞斯力
componentName_S='Mises'
#设置后处理处理的场量与分量
vw.odbDisplay.setPrimaryVariable(str(fieldName_S),field1.position,1,str(componentName_S))
#输出应力结果图片
vw.printPNGOffScreen("mises.png")

```

2.3 界面二次开发

开发人员可基于标准模板开发 Python 界面，并通过动态添加的方式扩展到 SAM 软件中。

根据功能需要，软件功能界面以模块（Module）的方式注册到主界面中，每个模块中包含有对应的菜单栏和工具栏。

2.3.1 启动项

打开 SAM 软件安装目录下\Release，可通过右键点击桌面快捷方式图标，在右键菜单中点击“打开文件所在位置”。



图 21 打开启动项目录

在启动项目录下找到批处理文件 samAPP.bat。

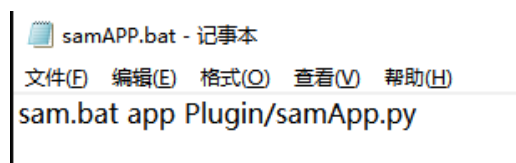


图 22 批处理脚本内容

此脚本内容含义为：

第一个参数 sam.bat，执行启动目录下 sam.bat 批处理脚本；

第二个参数 **app**，使用 **app** 模式启动 **SAM** 软件，此模式下会根据第三个参数加载初始化脚本；

第三个参数指定初始化脚本为启动项目目录下 **Plugin/samApp.py**。

2.3.2 通过脚本初始化需要加载的模块

界面二次开发初始化脚本为启动项目目录下 **Plugin/samApp.py**。

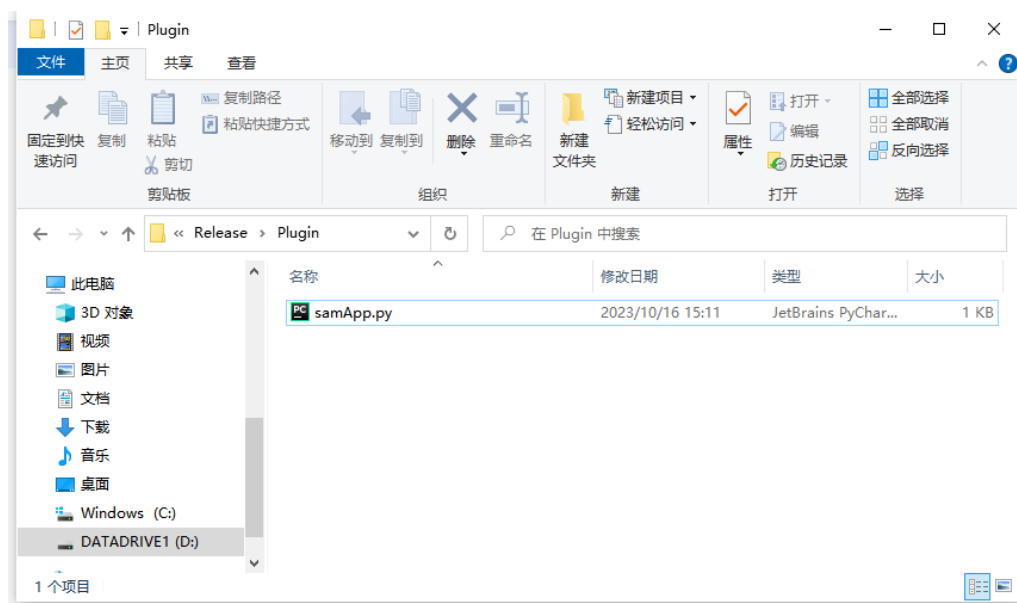


图 23 初始化脚本

脚本内容如下：

#引用 **pythonos** 库和 **sys** 库

#**sys** 模块负责处理 **Python** 运行时的配置及资源，从而可以与系统环境交互

#**os** 模块负责程序与操作系统的交互，提供了访问操作系统底层的接口

```
import os, sys
```

```
sys.path.append(os.path.abspath('__file__'))
```

#引用 **SAM** 的界面库

```
from FEGuiPy import *
```

#初始化 **SAM** 程序

```
app = FEApp()
```

```
app.init(sys.argv)
```

#初始化 **SAM** 主界面

```
mw = FEMainWindow(app)
```



```

#注册 SAM 界面模块，默认为 SAM 自带的基础模块
mw.registerModule("Part","Part")
mw.registerModule("Property","PropertyModuleGui")
mw.registerModule("Assembly","AssemblyModuleGui")
mw.registerModule("Step","StepModuleGui")
mw.registerModule("Interaction","InteractionModuleGui")
mw.registerModule("Load","LoadModuleGui")
mw.registerModule("Mesh","MeshModuleGui")
mw.registerModule("Job","JobModuleGui")
mw.registerModule("Visualization","VisualizationModuleGui")
#创建 SAM 程序
app.create()
#执行 SAM 程序
app.run()

```

通过上述脚本内容可看出，**SAM** 可通过在初始化脚本中注册模块的方式将开发的界面添加到程序主界面中，其注册语句 **registerModule** 需要两个参数，第一个参数为注册的模块名称，第二个参数为注册的模块类。

开发人员可以在注册模块语句中加入自己想要注册的模块信息，语句的顺序决定了软件打开后界面上模块的排布顺序。

我们在 **Visualization** 模块后添加一个测试示例模块：

```
mw.registerModule("CustomModule","CustomModuleGui")
```

添加完语句后，软件在启动时会在初始化脚本所在目录（启动项目目录/**Plugin**）下搜索模块内容，因此我们要在此目录下编写对应模块内容，否则软件无法启动。具体模块内容格式请参看下一章节。

2.3.3 模块

SAM 的界面模块包含界面定义与语句。

在软件安装目录下 **Example/SecondaryDevelopment** 文件夹中附有一个界面模块示例脚本 **CustomModuleGui.py**，我们将此脚本拷贝到软件启动项目目录下 **Plugin** 目录。

```

from FEGuiPy import *
from PySide2.QtWidgets import QMenu, QMessageBox
from PySide2.QtCore import QObject, Signal, Slot

class CustomToolsetGui(FEToolsetGui):
    def __init__(self):
        FEToolsetGui.__init__(self, "Custom")
        menu = QMenu("Custom", self)
        menu.addAction("SayHello...", self.SayHello)

    @Slot()
    def SayHello(self):
        box = QMessageBox()
        box.warning(self, "Custom", "Hello World!")

customToolset = CustomToolsetGui()

class CustomModuleGui(FEModuleGui):
    def __init__(self):
        FEModuleGui.__init__(self, "CustomModule", FEModuleGui.PART)
        self.registerToolset(customToolset, GUI_IN_MENUBAR)

customModule = CustomModuleGui()

```

图 24 界面模块示例

下文我们将介绍如何编写界面模块内容。

2.3.3.1 引用

界面模块需要引用的库一般分为 2 类：

- SAM 提供的动态库，如界面库 FEGuiPy 和数据访问库 kernelAccess；
- Qt 提供的 PySide 库，比较常用的有 PySide2.QtWidgets、PySide2.QtCore、PySide2.QtGui 等。

我们可以根据界面需求来决定引用哪些库。本示例中我们需要在界面上实现菜单栏及按钮，点击对应按钮会执行相应的命令，因此我们需要引用的库如下：

```

from FEGuiPy import *
from PySide2.QtWidgets import QMenu, QMessageBox
from PySide2.QtCore import QObject, Signal, Slot
from samConstants import *

```

2.3.3.2 模块类

界面模块的添加需要定义一个模块类，在 SAM 中界面模块类继承自模板类 FEModuleGui，名称与启动项目目录下 Plugin/samApp.py 中注册的模块类名称（第二个参数）相同。其定义如下：

```
class CustomModuleGui(FEModuleGui):
def __init__(self):
FEModuleGui.__init__(self,"CustomModule",FEModuleGui.PART)
```

```
customModule=CustomModuleGui()
```

在类初始化函数中，调用父类的初始化函数，传入 3 个参数：

- **self**，模块类对象；
- 注册的模块名称，与启动项目目录下 **Plugin/samApp.py** 中注册的模块名称（第一个参数）相同；
- 显示对象，规定此模块下模型显示类型，格式为 **FEModuleGui.Type**，其中 **Type** 为枚举值，包含 **PART**（部件显示）、**ASSEMBLY**（装配体显示）、**ODB**（后处理显示）3 种类型。

完成类定义后，我们保存文件，双击 **SAM** 启动项目目录下的批处理脚本 **samAPP.bat**，可查看模块类添加的效果，如图所示，模块下拉框中已经添加了我们定义的模块。

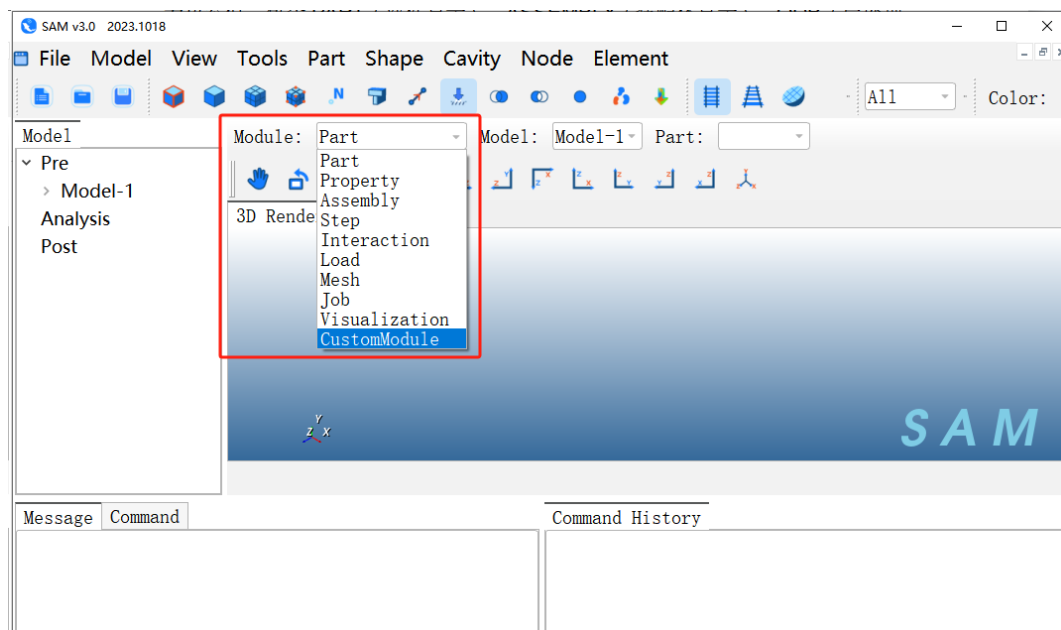


图 25 添加示例模块

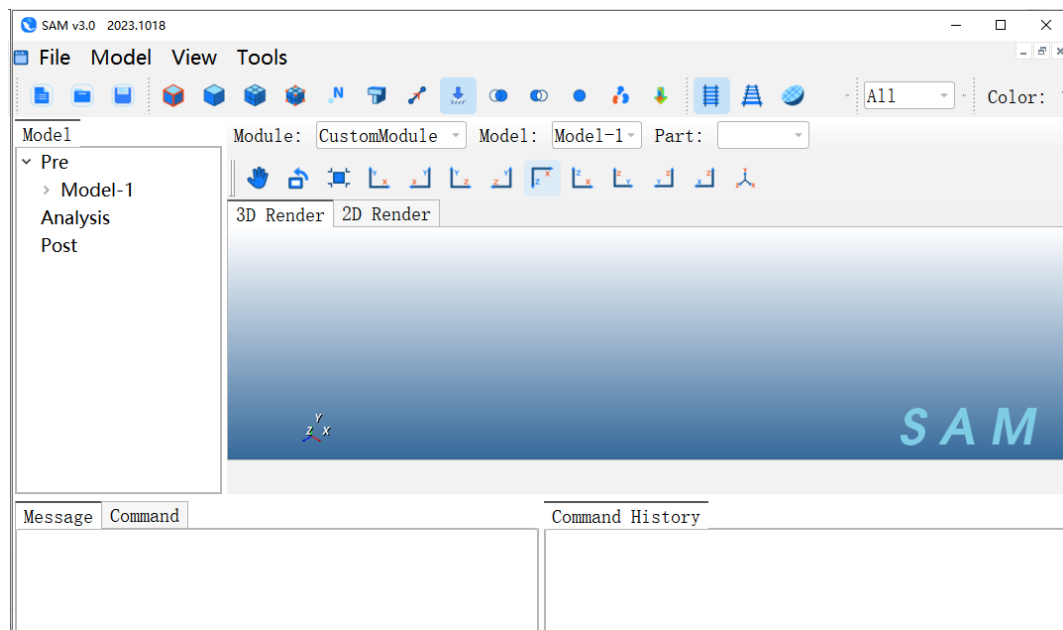


图 26 切换到示例模块

下面我们将介绍如何给示例模块中添加自定义菜单栏、工具栏等内容。

2.3.3.3 菜单栏及工具栏

要实现自定义菜单栏、工具栏的创建，需要定义一个菜单栏类，在 **SAM** 中继承自模板类 **FEToolsetGui**。

```
class CustomToolsetGui(FEToolsetGui):
```

```
def __init__(self):
```

```
FEToolsetGui.__init__(self, "Custom")
```

#定义类对象

```
customToolset = CustomToolsetGui()
```

在类初始化函数中，调用父类的初始化函数，传入 2 个参数：

- **self**，菜单栏类对象；
- 类名称。

完成初始化后，具体的界面设计与槽函数定义写在我们定义的菜单栏中：

```
class CustomToolsetGui(FEToolsetGui):
```

```
def __init__(self):
```

```
FEToolsetGui.__init__(self, "Custom")
```

```
menu = QMenu("Custom", self)
```

```
menu.addAction("SayHello...", self.SayHello)
```

```
@Slot()
```

```
defSayHello(self):
```

```
box=QMessageBox()
```

```
box.warning(self,"Custom","HelloWorld!")
```

✧ 由于语法规则，菜单栏类定义和对象定义需要写在模块类定义之前。

要在模块中添加我们定义的菜单栏及工具栏，需要在模块类中添加一行代码：

```
self.registerToolset(customToolset,GUI_IN_MENUBAR)
```

接口函数需要传入 2 个参数：

- 类对象；
- 界面类型，用枚举值表示需要向界面中添加何种类型的界面元素，常用的有 **GUI_IN_MENUBAR**（菜单栏）和 **GUI_IN_TOOLBAR**（工具栏）。

完成类定义后，我们保存文件，双击 **SAM** 启动项目目录下的批处理脚本 **samAPP.bat**，可查看菜单栏添加的效果，如图所示：

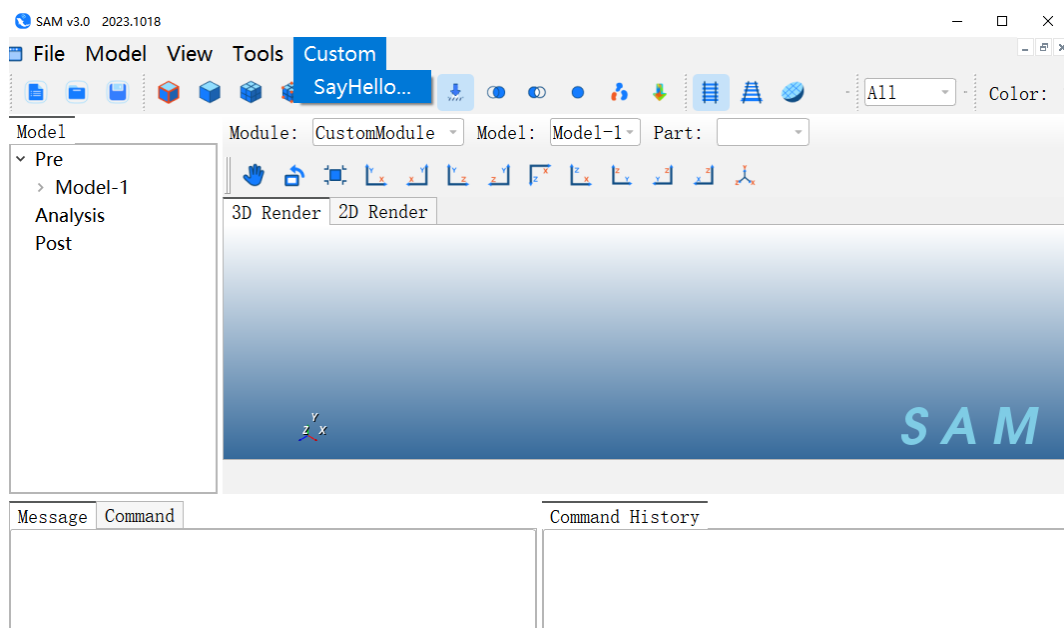


图 27 模块下自定义菜单栏

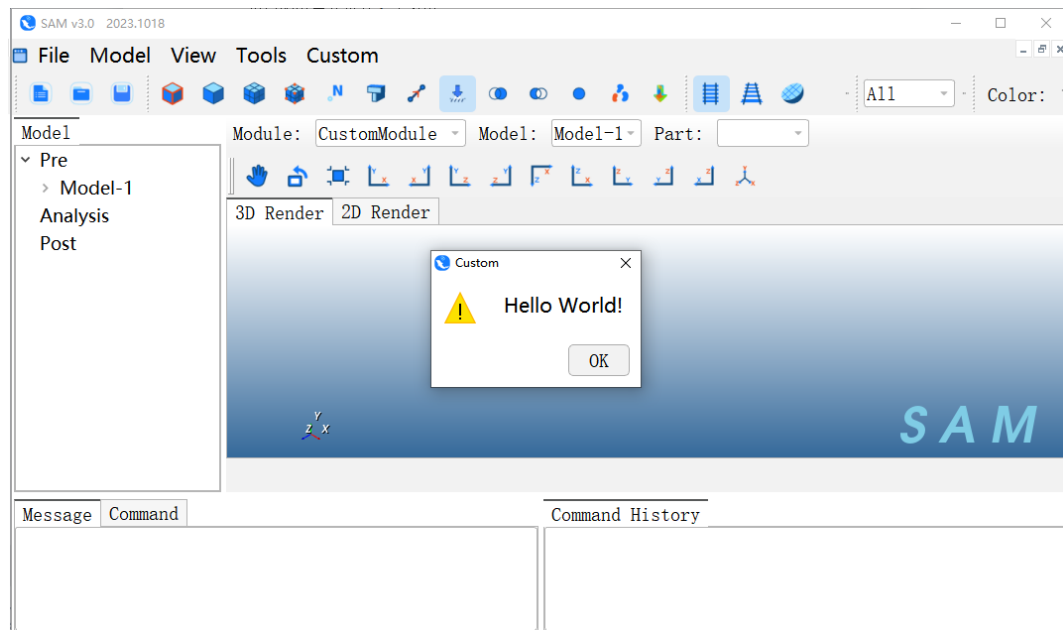


图 28 点击自定义按钮

2.3.3.4 发送内核命令

我们可以在界面模块中使用 SAM 的 Python 接口向 SAM 内核发送命令，以实现对内核数据的操作。

我们在自定义的菜单栏中新增一个按钮，向内核发送一个参数化建模的命令（高亮部分代码）。

```
class CustomToolsetGui(FEToolsetGui):
    def __init__(self):
        FEToolsetGui.__init__(self, "Custom")
        menu = QMenu("Custom", self)
        menu.addAction("SayHello...", self.SayHello)
        menu.addAction("CreateCylinder", self.CreateCylinder)
```

```
@Slot()
def SayHello(self):
    box = QMessageBox()
    box.warning(self, "Custom", "HelloWorld!")
```

```
@Slot()
```

```
defCreateCylinder(self):
fromkernelAccessimportmdb
p=mdb.models['Model-1'].parts['Part-1']
p.BaseParametricCylinder(5,10,360,5,15,OFF)
```

保存文件后，双击 **SAM** 启动项目目录下的批处理脚本 **samAPP.bat**，打开软件。

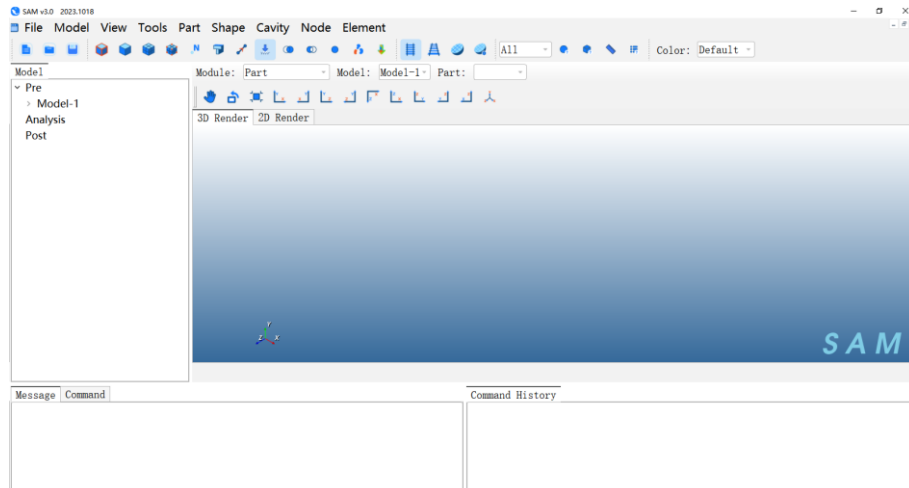


图 29 打开软件

点击 **Part->Create**，打开 **Part** 创建窗口，点击 **OK**，完成 **Part** 的创建。

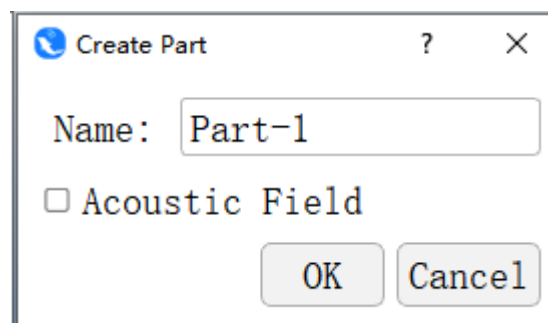


图 30 创建 Part

然后在 **Module** 下拉框中，切换到我们创建的 **CustomModule** 模块。

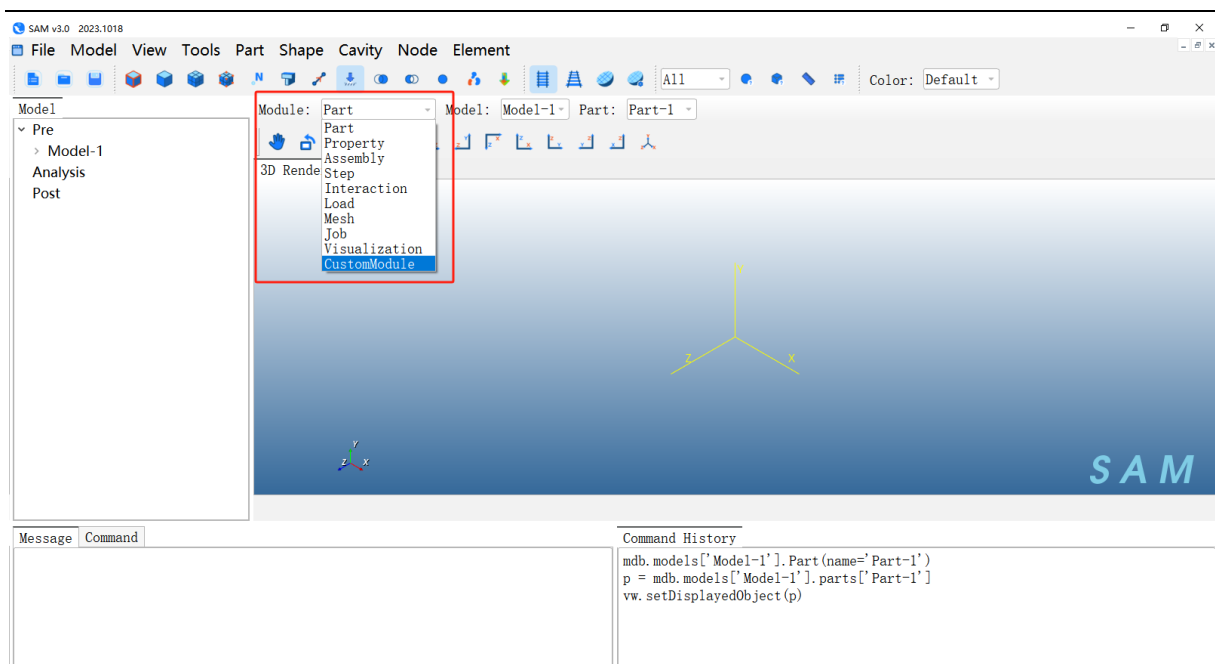


图 31 创建 Part

点击 Custom->CreateCylinder，即可发送我们设置好的命令，可以看到界面上创建了一个开口圆柱壳网格模型。

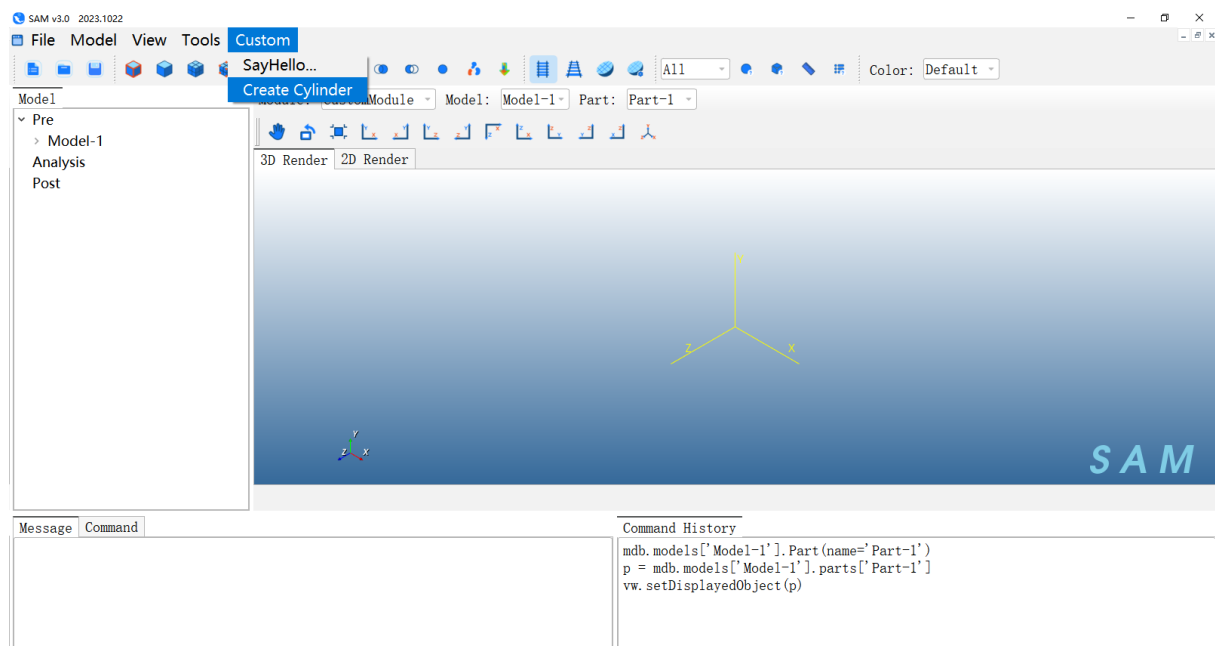


图 32 点击按钮

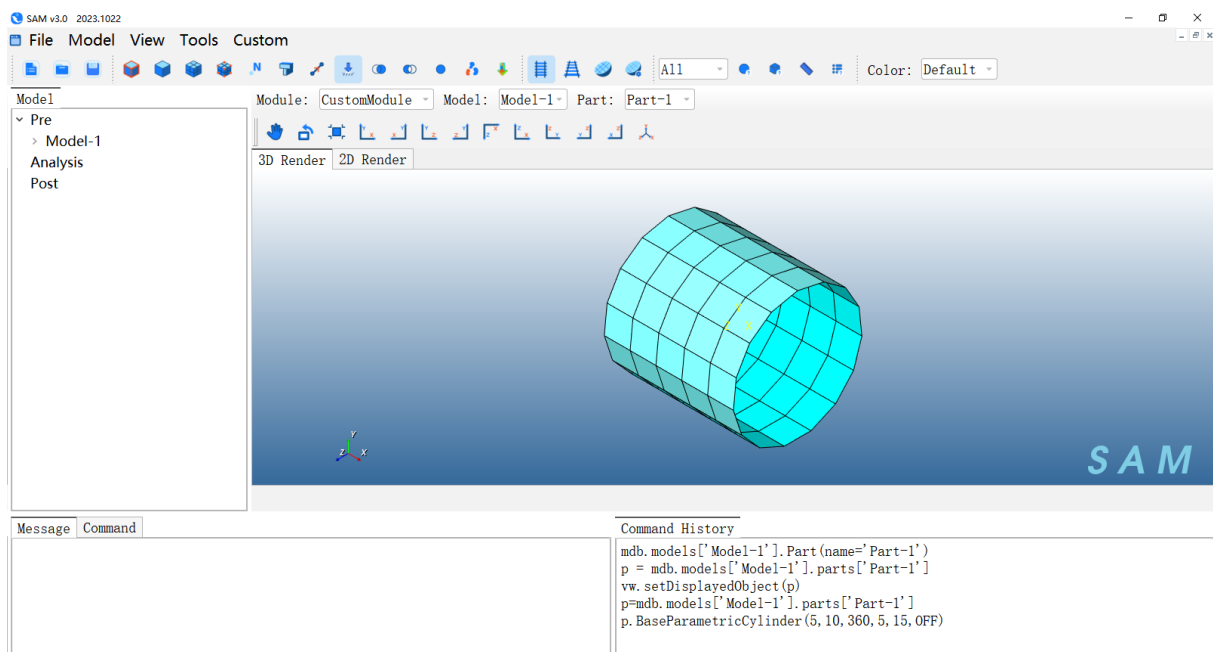


图 33 发送命令，创建网格模型

3 接口说明

3.1 Session 内置方法

3.1.1 Viewport()

(1) 作用描述：创建或获取一个 CAE 视口对象

(2) 示例：

```
session.Viewport(name='Viewport:1',origin=(0.0,0.0),width=290.5546875,height=117.944450378418)
```

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	'Viewport:1'	视口名称。若已存在则返回该对象，否则创建新视口
origin	tuple	是	(0.0,0.0)	视口左下角屏幕坐标(x,y)，单位像素
width	Float	是	窗口宽度	视口宽度，单位像素，最小值≥100
height	Float	是	窗口高度	视口高度，单位像素，最小值≥100

(4) 输出参数：

返回值	类型	说明
-----	----	----

viewport	Viewport	视口对象，后续可调用 <code>makeCurrent()</code> 、 <code>maximize()</code> 等方法
----------	----------	---

3.1.2 Viewport.makeCurrent()

(1) 作用描述：将指定视口设为当前活动视口，后续显示操作默认作用于此视口

(2) 示例：`session.viewports['Viewport:1'].makeCurrent()`

(3) 输入参数：

参数	类型	必需	默认值	说明
无	—	—	—	本方法无输入参数

(4) 输出参数：

返回值	类型	说明
无	None	无返回值。执行后当前活动视口被切换

3.1.3 Viewport.maximize()

(1) 作用描述：最大化指定视口

(2) 示例：`session.viewports['Viewport:1'].maximize()`

(3) 输入参数：

参数	类型	必需	默认值	说明
无	—	—	—	本方法无输入参数

(4) 输出参数：

返回值	类型	说明
无	None	无返回值。无返回值。视口被最大化

3.1.4 Viewport.partDisplay.setValues()

(1) 作用描述：设置视口内零件（Part）模块的可视化显示开关

(2) 示例：

```
session.viewports['Viewport:1'].partDisplay.setValues(sectionAssignments=ON,engineeringFeatures=ON)
```

```
session.viewports['Viewport:1'].partDisplay.setValues(renderBeamProfiles=ON)
```

```
session.viewports['Viewport:1'].partDisplay.setValues(mesh=ON)
```

(4) session.viewports['Viewport:1'].partDisplay.meshOptions.setValues(
meshTechnique=ON)

(3) 输入参数:

参数	类型	默认值	说明
sectionAssignments	Boolean	ON	控制是否在零件视图中渲染截面分配标记; 开启后壳/梁截面、实体截面赋值区域会显示色块标识
engineeringFeatures	Boolean	ON	控制是否显示零件的工程修饰特征, 包含圆角、倒角、切口、抽壳、筋板等成型工艺特征
mesh	Boolean	ON	是否显示网格
renderBeamProfiles	Boolean	ON	是否渲染梁截面形状
meshOptions	dict	ON	网格显示选项, 含 meshTechnique

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。显示属性被更新

3.1.5 Viewport.partDisplay.meshOptions.setValues()

(1) 作用描述: 控制零件视图中**网格划分算法标识**的显示与隐藏, 开启后模型不同区域会显示对应网格算法的标记色块/线条, 用于直观区分模型各区域采用的网格生成方式

(2) 示例:

```
session.viewports['Viewport:1'].partDisplay.meshOptions.setValues(meshTechnique=ON)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
meshTechnique	Boolean	否	OFF	ON=自由网格,OFF=结构化网格
technique	Symbol	否	FREE	FREE/STRUCTURED/SWEEP/BOTTOM_UP/SYSTEM

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。网格显示技术被更新

3.1.6 Viewport.setValues()

(1) 作用描述: 修改视图窗口 (Viewport) 属性

(2) 示例: `session.viewports['Viewport:1'].setValues(displayedObject=p)`

(3) 输入参数:

参数	类型	必需	默认值	说明
displayedObject	Object	是	—	要显示的对象: Part/Assembly/Odb 等

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。视口显示对象被更新

3.1.7 Viewport.view.fitView()

(1) 作用描述: 自动调整视图以显示全部对象

(2) 示例: `session.viewports['Viewport:1'].view.fitView()`

3.1.8 Viewport.view.setValues()

(1) 作用描述：调整视图对象

(2) 示例：`session.viewports['Viewport:1'].view.setValues(session.views['Front'])`

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	视图名称： 'Front','Right','Top','Back','Left','Bottom' '等

(4) 输出参数：

返回值	类型	说明
无	None	无返回值。视图方向被切换

3.1.9 Viewport.view.setProjection()

(1) 作用描述：设置视图的透视

(2) 示例：`session.viewports['Viewport: 1'].view.setProjection(projection=PARALLEL)`

(3) 输入参数：

参数	类型	必需	默认值	说明
projection	SymbolicConstant	是	—	透视方式 (PARALLEL/PERSPECTIVE))

(4) 输出参数：

返回值	类型	说明
无	None	无返回值。

3.1.10 Viewport.assemblyDisplay.setValues()

(1) 作用描述：

(2) 示例：

```
session.viewports['Viewport:1'].assemblyDisplay.setValues(renderBeamProfiles=OFF,optimizationTasks=OFF,geometricRestrictions=OFF,stopConditions=OFF,adaptiveMeshConstraints=ON,loads=ON,bcs=ON,predefinedFields=ON,connectors=ON,mesh=OFF)
```

(3) 输入参数：

参数	类型	必需	默认值	说明
renderBeamProfiles	Boolean	否	ON	渲染梁截面
optimizationTasks	Boolean	否	OFF	显示优化任务
geometricRestrictions	Boolean	否	OFF	显示几何约束
stopConditions	Boolean	否	OFF	显示停止条件
loads	Boolean	否	OFF	显示载荷
bcs	Boolean	否	OFF	显示边界条件
predefinedFields	Boolean	否	OFF	显示预定义场
connectors	Boolean	否	OFF	显示连接器
mesh	Boolean	否	OFF	显示网格

(4) 输出参数：

返回值	类型	说明
无	None	无返回值。装配体显示属性被更新

3.1.11 Viewport.odbDisplay.setFrame()

(1) 作用描述：设置 ODB 显示的分析步和帧

(2) 示例：`session.viewports['Viewport:1'].odbDisplay.setFrame(step=0,frame=10)`

(3) 输入参数：

参数	类型	必需	默认值	说明
step	Int	否	0	分析步编号（0=Initial）
frame	Int	否	0	帧号。频率步中为特征值序号

(4) 输出参数：

返回值	类型	说明
无	None	无返回值。ODB 帧被切换

3.1.12 Viewport.odbDisplay.display.setValues()

(1) 作用描述：设置 ODB 显示状态

(2) 示例：

```
session.viewports['Viewport:1'].odbDisplay.display.setValues(plotState=(CONTOURS_ON_DEF,))
```

(3) 输入参数：

参数	类型	必需	默认值	说明
plotState	tuple	否	(DEFORMED,)	显示状态云图： CONTOURS_ON_DEF

(4) 输出参数：

返回值	类型	说明
无	None	无返回值。显示状态被更新

3.1.13 Viewport.odbDisplay.setPrimaryVariable()

(1) 作用描述：设置 ODB 主显示变量

(2) 示例：

```
session.viewports['Viewport:1'].odbDisplay.setPrimaryVariable(  
variableLabel='U',outputPosition=NODAL,refinement=(1,'Magnitude'))
```

(3) 输入参数：

参数	类型	必需	默认值	说明
variableLabel	String	是	—	变量标签：'U'(位移)/'S'(应力)/'E'(应变)/'RF'(反力)等
outputPosition	Symbol	否	NODAL	NODAL/INTEGRATION_POINT
refinement	tuple	否	(0,'')	(invariant,label)，如 (1,'Magnitude')表示显示分量

(4) 输出参数：

返回值	类型	说明
无	None	无返回值。主变量被设置

3.2 mdb 内置方法

3.2.1 mdb.models

3.2.1.1 mdb.models[name].Part()

(1) 作用描述：在指定模型中创建新部件

(2) 示例: `p=mdb.models['Model-1'].Part(name='Part-1')`

`p = mdb.models['Model-1'].parts['Part-1']`

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	Part-1	部件名称

(4) 输出参数:

返回值	类型	说明
part	Part 对象	创建的部件对象

3.2.1.2 mdb.PartFromStep()

(1) 作用描述: 从 STEP 文件导入几何并生成部件

(2) 示例:

`mdb.models['Model-1'].PartFromStep(filePath='D://bushing.stp', globalElemSize=6.0, isQuadratic=0)`

(3) 输入参数:

参数	类型	必需	默认值	说明
filePath	String	是	—	STEP 文件路径
globalElemSize	Float	否	—	全局单元尺寸
isQuadratic	Int	否	0	0=线性单元, 1=二次单元

(4) 输出参数:

返回值	类型	说明
Part	Part	部件对象

3.2.1.3 mdb.ModelFromNastranInputFile()

(1) 作用描述：从 **Nastran** 格式文件（.bdf）导入模型到本软件

(2) 示例：

```
mdb.ModelFromNastranInputFile(  
    inputFileName='model.bdf',  
    name='ImportedModel',  
    cbar='B33',  
    ctri3='S3I',  
    cquad4='S4I',  
    AutoProperty='False')
```

(3) 输入参数：

参数	类型	必需	默认值	说明
inputFileName	String	是	—	Nastran 输入文件路径
name	String	是	—	导入后的模型名称
cbar	String	否	'B33'	杆单元类型标识
ctri3	String	否	'S3I'	三角形壳单元类型标识
cquad4	String	否	'S4I'	四边形壳单元类型标识
AutoProperty	String	否	'False'	是否使用 bdf 中的材料属性， 'True'使用 Bdf 中的材料命名， 'False'转换为本软件的材料命名机制

(4) 输出参数：

返回值	类型	说明
无	None	无返回值。模型被添加到 mdb.models 字典

3.2.1.4 mdb.ModelFromInputFile()

(1) 作用描述：从外部文件导入模型.inp

(2) 示例：

```
mdb.ModelFromInputFile(  
    name='Sign',  
    inputFileName='D:/Program  
Files/SAM/Example/ReponseSpectrumSign/Sign.inp')
```

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	模型名称
inputFileName	String	是	—	输入文件路径（.inp 等）

(4) 输出参数：

返回值	类型	说明
Model 对象	Model	返回导入的模型

3.2.2 Part()

3.2.2.1 Model.Part()

(1) 作用描述：创建零件对象

(2) 示例：p=mdb.models['Model-1'].Part(name='Part-1')

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	零件名称

(4) 输出参数：

返回值	类型	说明
part	Part	零件对象，后续可用 .BaseParametricStiffenedPlate()、.elements 等

3.2.2.2 Part.BaseParametricRectangle()

- (1) 作用描述：创建参数化长方体
- (2) 示例：

```
p.BaseParametricRectangle(length=5, width=5, lengthScale=5, widthScale=5)
```

- (3) 输入参数：

参数	类型	必需	默认值	说明
length	Float	否	1.0	X 方向长度
width	Float	否	1.0	Y 方向宽度
lengthScale	Float	否	1.0	X 方向网格密度
widthScale	Float	否	1.0	Y 方向网格密度

- (4) 输出参数：

返回值	类型	说明
Part 对象	Part	返回创建的部件（含网格）

3.2.2.3 Part.BaseParametricRectangleWithHole()

- (1) 作用描述：创建带孔的参数化矩形几何
- (2) 示例：

```
mdb.models['Model-1'].parts['Part-1'].BaseParametricRectangleWithHole(length=10, width=5, radius=0.2, nx=20, ny=10, nCircle=32, enableTriElement='OFF')
```

- (3) 输入参数：

参数	类型	必需	默认值	说明
length	float	是	-	矩形长度（X 方向）
width	float	是	-	矩形宽度（Y 方向）
radius	float	是	-	圆角/孔半径
nx	int	是	-	X 方向单元分割数
ny	int	是	-	Y 方向单元分割数
nCircle	int	是	-	圆弧/孔的分段数
enableTriElement	str	否	'OFF'	是否允许三角形单元： 'ON'/'OFF'

（4）输出参数：

返回值	类型	说明
feature	PartFeature 对象	创建的几何特征对象

3.2.2.4 Part.BaseParametricCylinder ()

（1）作用描述：创建参数化圆柱体部件

（2）示例：

p.BaseParametricCylinder(radius=5, height=10, angle=360, axial=5, circular=40, closed=False)

（3）输入参数：

参数	类型	必需	默认值	说明
radius	Float	是	5	半径
height	Float	是	10	高度

参数	类型	必需	默认值	说明
angle	Float	是	360	圆心角（度）
axial	Int	是	5	轴向单元数
circular	Int	是	40	圆周单元数
closed	Boolean	是	False	是否封闭两端

（4）输出参数：

返回值	类型	说明
PartFeature	PartFeature	部件特征对象

3.2.2.5 Part.BaseParametricStiffenedPlate()

（1）作用描述：创建参数化加筋板（底板+加筋条）

（2）示例：

```
mdb.models['Model-1'].parts['Part-1'].BaseParametricStiffenedPlate(  
XBarList='0,500,1000,1500,2000',YBarList='0,2000',  
insertPoint='0,0,0',startVector='0,0,0',endVector='0,0,0',  
angle=0,includePlate=1,addBeamsOnTheSides=0,  
tableXNum=5,tableYNum=2,modelName='Model-1',partName='Part-1',  
thickness=10,beamXOrientation='0,1,0',beamYOrientation='-1,0,0',  
profileX='Profile-1',profileY='Profile-1',materialName='Material-1')
```

（3）输入参数：

参数	类型	必需	默认值	说明
XBarList	String	是	—	X 方向加劲条位置， 逗号分隔
YBarList	String	是	—	Y 方向加劲条位置， 逗号分隔

参数	类型	必需	默认值	说明
insertPoint	String	是	'0,0,0'	插入点坐标(x,y,z)
startVector	String	是	'0,0,0'	起始方向向量
endVector	String	是	'0,0,0'	终止方向向量
angle	Float	否	0	旋转角度（度）
includePlate	Int	否	1	是否包含底板（1=是,0=否）
addBeamsOnTheSides	Int	否	0	是否在边缘添加梁（1=是,0=否）
tableXNum	Int	是	—	X 方向分段数
tableYNum	Int	是	—	Y 方向分段数
modelName	String	是	—	模型名称
partName	String	是	—	零件名称
thickness	Float	是	—	板厚度
beamXOrientation	String	是	—	X 方向梁的局部 1 轴方向
beamYOrientation	String	是	—	Y 方向梁的局部 1 轴方向
profileX	String	是	—	X 方向梁截面名称
profileY	String	是	—	Y 方向梁截面名称
materialName	String	是	—	材料名称

（4）输出参数：

返回值	类型	说明
无	None	无返回值。加筋板零件被创建

3.2.2.6 Part.BaseParametricCuboid()

(1) 作用描述：创建参数化长方体

(2) 示例：

```
p.BaseParametricCuboid(length=0.1, width=0.1, height=0.02, meshSize=0.005)
```

(3) 输入参数：

参数	类型	必需	默认值	说明
length	Float	否	1.0	X 方向长度
width	Float	否	1.0	Y 方向宽度
height	Float	否	1.0	Z 方向高度
meshSize	Float	否	1.0	网格尺寸（越小越密）

(4) 输出参数：

返回值	类型	说明
Part 对象	Part	返回创建的部件（含网格）

3.2.2.7 Part.Sets

(1) 作用描述：在部件上创建集合（Set）

(2) 示例：

```
region = p.Set(elements=elements1, name='Set-1')
```

(3) 输入参数：

参数	类型	必需	默认值	说明
elements	Sequence[Int]	否	—	单元集合

参数	类型	必需	默认值	说明
nodes	Sequence[Int]	否	—	节点集合
faces	Sequence[Int]	否	—	面集合
edges	Sequence[Int]	否	—	边集合
name	String	是	—	集合名称
setType	SymbolicConstant	否	—	集合类型

(4) 输出参数:

返回值	类型	说明
Set	Set	集合对象

3.2.2.8 Part.engineeringFeatures

(1) 作用描述: 获取零件的工程特征集合 (只读属性)

(2) 示例: `region=mdb.models['Model-1'].parts['Part-1'].allSets['Set-shell']`

(3) 输入参数:

参数	类型	必需	默认值	说明
无	—	—	—	只读属性, 无输入参数

(4) 输出参数:

返回值	类型	说明
engineeringFeatures	EngineeringFeatures	工程特征集合, 可调用 <code>.NonstructuralMass()</code> 等

3.2.2.9 Part.EngineeringFeatures.NonstructuralMass()

(1) 作用描述：添加非结构质量

(2) 示例：

```
mdb.models['Model-1'].parts['Part-1'].engineeringFeatures.NonstructuralMass(  
    name='Inertia-1',region=region,units=TOTAL_MASS,  
    magnitude=1.0,distribution=MASS_PROPORTIONAL)
```

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	名称
region	Region	是	—	作用区域
units	Symbol	是	—	TOTAL_MASS/MASS_PER_AREA/MASS_PER_LENGTH
magnitude	Float	是	—	质量值
distribution	Symbol	否	UNIFORM	MASS_PROPORTIONAL/UNIFORM

(4) 输出参数：

返回值	类型	说明
nonstructuralMass	NonstructuralMass	非结构质量对象

3.2.2.10 ElementArray.getSequenceFromMask()

(1) 作用描述：按掩码选取单元

(2) 示例：

```
p = mdb.models['Model-1'].parts['Part-1']
```

```
elements1= p.elements.getSequenceFromMask(mask=['#7f'],))
```

(3) 输入参数:

参数	类型	必需	默认值	说明
mask	String	是	—	元素掩码, 十六进制位表示。如 '#7f' 表示选择特定单元

(4) 输出参数:

返回值	类型	说明
Sequence[Int]	Sequence	单元编号序列

3.2.2.11 nodeArray.getSequenceFromMask()

(1) 作用描述: 通过掩码字符串筛选节点编号序列

(2) 示例:

```
a = mdb.models['Model-1'].rootAssembly
n1 = a.instances['Part-1-1'].nodes
nodes1 = n1.getSequenceFromMask(mask=(
    ['#41041041 #10410410 #4104104 #41041041 #10410410 #4104104
#41041041',
    '#410'], ))
```

(3) 输入参数:

参数	类型	必需	默认值	说明
mask	Sequence[String]	是	—	掩码字符串, 用于筛选节点

(4) 输出参数:

返回值	类型	说明
Sequence[Int]	Sequence	节点编号序列

3.2.3 Material()

- (1) 作用描述：创建材料对象
- (2) 示例：`mdb.models['Model-1'].Material(name='Material-1')`
- (3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	材料名称
description	String	否	''	材料描述
materialType	Symbol	否	MECHANICAL	材料类型：Density / Elastic / Plastic 等

- (4) 输出参数：

返回值	类型	说明
material	Material	材料对象，后续可调用.Density()、.Elastic()等方法

3.2.3.1 Material.Density()

- (1) 作用描述：定义材料密度
- (2) 示例：`mdb.models['Model-1'].materials['Material-1'].Density(table=((7.8e-09,)),)`
- (3) 输入参数：

参数	类型	必需	默认值	说明
table	sequence	是	—	密度值序列。单位由模型单位制决定。单值时((value,)), 多值时

参数	类型	必需	默认值	说明
				[(value1),(value2),...]
massPerVolume	Float	否	None	质量密度，与 table 互斥

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。密度属性被添加到材料

3.2.3.2 Material.Elastic()

(1) 作用描述:

```
mdb.models['Model-1'].materials['Material-1'].Elastic(
type=ISOTROPIC,table=((210000.0,0.3),),noCompression=False)
```

(2) 示例:

```
mdb.models['Model-1'].materials['Material-1'].Elastic(
type=ISOTROPIC,table=((210000.0,0.3),),noCompression=False)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
type	Symbol	是	—	弹性类型: ISOTROPIC/ORTHOTROPIC/ANISOTROPIC/ENGINEERING_CONSTANTS/LAMINA/PLANAR/TRACTION/USER
table	sequence	是	—	弹性参数表。ISOTROPIC 时为(E,v)
noCo	Bo	否	F	仅 ORTHOTROPIC 有效，是否允许压缩

参数	类型	必需	默认值	说明
mpres sion	ole an		al se	

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。弹性属性被添加到材料

3.2.3.3 Material.Plastic()

(1) 作用描述: 定义材料塑性属性

(2) 示例:

```

mdb.models['Model-1'].materials['Material-1'].Plastic(
    table=((235000000.0, 0.0), (314000000.0, 0.1),
           (372000000.0, 0.2), (411000000.0, 0.3)))

```

(3) 输入参数:

参数	类型	必需	默认值	说明
table	sequence	是	—	塑性数据表: [(应力 1, 塑性应变 1), (应力 2, 塑性应变 2), ...]
yieldStress	Float	否	None	屈服应力 (rate≠STATIC 时)
referenceS trainRate	Float	否	1.0	参考应变率 (rate≠STATIC 时)
hardening	Symbolic Constant	否	ISOTR OPIC	硬化类型: ISOTROPIC/KINEMATIC/JOH NSON_COOK/COMBINED
table	sequence	否	0	场变量依赖表 (dependencies>1 时)

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。塑性属性被添加到材料

3.2.4 Section()

3.2.4.1 Model.HomogeneousShellSection()

(1) 作用描述: 创建均匀厚度的壳截面

(2) 示例:

```
mdb.models['Model-1'].HomogeneousShellSection(name='Section-Shell',material='Material-1',thickness=0.1)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	-	截面名称
material	String	是	-	材料名称
thickness	float	是	-	壳厚度

(4) 输出参数:

返回值	类型	说明
section	ShellSection 对象	创建的壳截面对象

3.2.4.2 Model.HomogeneousSolidSection()

(1) 作用描述: 创建均质实体截面

(2) 示例:

```
mdb.models['Model-1'].HomogeneousSolidSection(name='Section-1',
material='Material-1')
```

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	—	截面名称
material	String	是	—	关联材料名

(4) 输出参数:

返回值	类型	说明
Section	Section	截面对象

3.2.4.3 Model.BeamSection()

(1) 作用描述: 定义梁截面属性

(2) 示例:

```
mdb.models['Model-1'].BeamSection(name='Section-1', profile='Profile-1',
integration=DURING_ANALYSIS, material='Material-1')
```

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	—	截面名称
profile	String	是	—	截面轮廓名称 (Profile)
integration	SymbolicConstant	否	DURING_ANALYSIS	积分方式
material	String	是	—	材料名称

参数	类型	必需	默认值	说明
description	String	否	"	描述

(4) 输出参数:

返回值	类型	说明
BeamSection 对象	BeamSection	返回创建的梁截面

3.2.5 Assign()

3.2.5.1 p.SectionAssignment()

(1) 作用描述: 将截面属性分配给指定的区域

(2) 示例: p.SectionAssignment(region=region,sectionName='Section-Shell')

(3) 输入参数:

参数	类型	必需	默认值	说明
region	Region	是	-	区域对象(通常由 Set 获得)
sectionName	String	是	-	截面名称
offset	float	否	0.0	偏移距离
offsetField	String	否		偏移场变量
offsetType	SymbolicConstant	否	MIDDLE_SURFACE	偏移类型

参数	类型	必需	默认值	说明
n1	Sequence	否	(0.0, 0.0, 0.0)	法向向量
n1field	String	否	"	法向场变量
offset1Type	SymbolicConstant	否	UNIFORM	偏移 1 类型： UNIFORM/FIELD
offset1Vector	Sequence	否	(0.0, 0.0, 0.0)	偏移 1 向量
offset1Field	String	否	"	偏移 1 场变量

(4) 输出参数:

返回值	类型	说明
无	-	无返回值

3.2.6 Profile()

3.2.6.1 Model.TProfile()

(1) 作用描述: 创建 T 型截面轮廓

(2) 示例:

```
mdb.models['Model-1'].TProfile(name='Profile-1',b=120.0,h=150.0,tf=10.0,tw=10.0,l=0.0)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	—	截面名称
b	Float	是	—	翼缘宽度

参数	类型	必需	默认值	说明
h	Float	是	—	截面总高度
tf	Float	是	—	翼缘厚度
tw	Float	是	—	腹板厚度
l	Float	否	0.0	倒角长度

(4) 输出参数:

返回值	类型	说明
profile	Profile	截面对象, 可用于 BeamSection 或 BaseParametricStiffenedPlate

3.2.6.1.1 Profile.setValues()

(1) 作用描述: 修改型材参数

(2) 示例:

```
mdb.models['Model-1'].profiles['T_211-41'].setValues(
    b=38.0, h=44.0, tf=5.0, tw=5.0, l=0.0)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
b	Float	否	—	翼缘宽度 (mm)
h	Float	否	—	截面总高度 (mm)
l	Float	否	0.0	圆角半径
tf	Float	否	—	翼缘厚度 (mm)
tw	Float	否	—	腹板厚度 (mm)

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。轮廓参数被更新

3.2.6.2 Model.GeneralizedProfile()

(1) 作用描述: 创建 **General** 梁截面轮廓 (用于自定义截面属性)

(2) 示例:

```
mdb.models['Model-1'].GeneralizedProfile(  
    name='Profile-1', area=1.0, i11=13333.33, i12=0.0, i22=1.0, j=0.0, gammaO=0.0,  
    gammaW=0.0)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	—	轮廓名称
area	Float	是	—	截面面积
i11	Float	是	—	惯性矩 I11
i12	Float	是	—	惯性积 I12
i22	Float	是	—	惯性矩 I22
j	Float	是	—	扭转常数 J
gammaO	Float	否	0.0	扇形坐标原点偏移
gammaW	Float	否	0.0	扇形坐标翘曲系数

(4) 输出参数:

返回值	类型	说明
Profile 对象	Profile	返回创建的轮廓对象

3.2.6.3 Model. RectangularProfileProfile()

(1) 作用描述：创建矩形梁截面轮廓

(2) 示例：

```
mdb.models['Model-1'].RectangularProfile(name='Profile-1', a=6.0, b=0.5)
```

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	轮廓名称
a	Float	是	—	截面高度（沿局部 2 轴）
b	Float	是	—	截面宽度（沿局部 1 轴）

(4) 输出参数：

返回值	类型	说明
Profile 对象	Profile	返回创建的轮廓对象

3.2.7 rootAssembly()

(1) 作用描述：获取模型的根装配体（只读属性）

(2) 示例：a=mdb.models['Model-1'].rootAssembly

(3) 输入参数：

参数	类型	必需	默认值	说明
无	—	—	—	只读属性，无输入参数

(4) 输出参数：

返回值	类型	说明
assembly	Assembly	根装配体对象，所有实例都挂载于此

3.2.7.1 Assembly.Instance()

(1) 作用描述：在装配体中创建零件实例

(2) 示例：`a.Instance(name='Part-1-1',part=p,dependent=OFF)`

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	实例名称
part	Part	是	—	要实例化的零件
dependent	Boolean	否	OFF	ON=从属实例（网格随零件更新），OFF=独立实例

(4) 输出参数：

返回值	类型	说明
instance	Instance	实例对象，后续可调用 <code>.nodes</code> 、 <code>.Set()</code> 等

3.2.7.2 Assembly.nodes

(1) 作用描述：获取实例的所有节点（只读属性）

(2) 示例：`n1=a.instances['Part-1-1'].nodes`

(3) 输入参数：

参数	类型	必需	默认值	说明
无	—	—	—	只读属性，无输入参数

(4) 输出参数：

返回值	类型	说明
nodes	NodeArray	节点数组对象，可调用 <code>.getSequenceFromMask()</code>

3.2.7.3 Assembly.Set()

(1) 作用描述：在装配体中创建集合

(2) 示例：a.Set(nodes=nodes1,name='Set-BC')

(3) 输入参数：

参数	类型	必需	默认值	说明
nodes	sequence	否	—	节点序列
elements	sequence	否	—	元素序列
faces	sequence	否	—	面序列
name	String	是	—	集合名称

(4) 输出参数：

返回值	类型	说明
set	Set	集合对象，可用于边界条件、载荷等

3.2.7.4 Assembly.Surface.()

(1) 作用描述：通过面单元创建表面

(2) 示例：

a.Surface(face1Elements=face1Elements1, face2Elements=face2Elements1,
name='Surf-1')

(3) 输入参数：

参数	类型	必需	默认值	说明
face1Elements	Sequence[Int]	否	—	第一组面单元
face2Elements	Sequence[Int]	否	—	第二组面单元
face3Elements	Sequence[Int]	否	—	第三组面单元

参数	类型	必需	默认值	说明
face4Elements	Sequence[Int]	否	—	第四组面单元
name	String	是	—	表面名称

(4) 输出参数:

返回值	类型	说明
Surface	Surface	表面对象

3.2.8 Step()

3.2.8.1 Model.StaticStep()

(1) 作用描述: 创建静力通用分析步

(2) 示例:

```
mdb.models['Model-1'].StaticStep(name='Step-1', previous='Initial',
    timePeriod=1.0, maxNumInc=100, initialInc=1.0, minInc=1e-05, maxInc=1.0,
    nlgeom=OFF, timeIncrementationMethod=AUTOMATIC)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	-	分析步名称
previous	String	是	-	前一步名称, 'Initial'表示初始步
timePeriod	float	否	1.0	分析步总时间
maxNumInc	int	否	100	最大增量步数

参数	类型	必需	默认值	说明
nlgeom	ON/OFF	否	OFF	是否开启几何非线性
initialInc	Float	否		初始增量步长
minInc	Float	否		最小增量步长
maxInc	Float	否		最大增量步长
timeIncrementationMethod	SymbolicConstant	否		增量控制方式 (AUTOMATIC/FIXED)

(4) 输出参数:

返回值	类型	说明
step	StaticStep 对象	创建的分析步对象

3.2.8.2 Model.FrequencyStep()

(1) 作用描述: 创建频率提取分析步

(2) 示例:

```

mdb.models['Model-1'].FrequencyStep(
    name='Step-1',previous='Initial',numEigen=10,
    eigensolver=LANCZOS,normalization=DISPLACEMENT,
    VMDistance=2.0,VMMatrix=FAST)

```

(3) 输入参数:

参数	类型	必需	默认值	说明
----	----	----	-----	----

参数	类型	必需	默认值	说明
name	String	是	—	分析步名称
previous	String	是	—	前一步名称, 'Initial' 表示第一步
numEigen	Int	否	1	提取的特征值数量
eigsolver	Symbol	否	LANCZOS	求解器: LANCZOS/SUBSPACE/AMS
normalization	Symbol	否	DISPLACEMENT	归一化方式: DISPLACEMENT/MASS
VMDistance	Float	否	0.0	广义特征值问题参考圆频率 ²
VMMatrix	Symbol	否	FAST	矩阵类型: FAST/General

(4) 输出参数:

返回值	类型	说明
step	Step	分析步对象

3.2.8.3 Model.BuckleStep()

(1) 作用描述: 创建屈曲分析步 (特征值屈曲)

(2) 示例:

```
mdb.models['Model-1'].BuckleStep(name='Step-1', previous='Initial',
eigsolver=LANCZOS, numEigen=4)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
----	----	----	-----	----

参数	类型	必需	默认值	说明
name	String	是	—	分析步名称
previous	String	是	—	前一步名称（首步用'Initial'）
numEigen	Int	否	10	提取的屈曲模态阶数
eigensolver	SymbolicConstant	否	LANCZOS	特征值求解器：LANCZOS/SUBSPACE
maxEigen	Float	否	0.0	特征值上限
minEigen	Float	否	0.0	特征值下限

（4）输出参数：

返回值	类型	说明
Step 对象	Step	返回创建的屈曲分析步对象

3.2.8.4 Model.SteadyStateDirectStep()

（1）作用描述：创建稳态动力分析步

（2）示例：

```

mdb.models['Model-1'].SteadyStateDirectStep(
    name='Step-1', previous='Initial',
    frequencyRange=((50, 1000, 20, 1), ),
    factorization=COMPLEX, scale=LINEAR,
    subdivideUsingEigenfrequencies=OFF)

```

（3）输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	步名称
previous	String	是	—	前一步名称
frequencyRange	Sequence	否	()	频率范围：((起始, 终止, 点数, 偏差),)
factorization	SymbolicConstant	否	REAL	矩阵分解： REAL/COMPLEX
scale	SymbolicConstant	否	LINEAR	缩放方式： LINEAR/LOGARITHMIC
subdivideUsingEigenfrequencies	Boolean	否	OFF	是否按固有频率细分
maxNumInc	Int	否	100	最大增量步数
initialInc	Float	否	1.0	初始增量
minInc	Float	否	1e-05	最小增量
maxInc	Float	否	1.0	最大增量

(4) 输出参数:

返回值	类型	说明
Step 对象	Step	返回创建的稳态动力分析步

3.2.8.5 Model.ResponseSpectrumStep()

(1) 作用描述: 创建响应谱分析步

(2) 示例:

```

mdb.models['Sign'].ResponseSpectrumStep(
    name='Step-2', previous='Step-1',
    comp=SINGLE_DIRECTION, sum=CQC,
    components= (('Amp-1', 0.0, 0.0, 1.0, 1.0, 0.0), ),
    directDamping=((1, 10, 1e-05), ))

```

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	—	步名称
previous	String	是	—	前一步名称
comp	SymbolicConstant	否	SINGLE_DIRECTION	SINGLE_DIRECTION (单向)
sum	SymbolicConstant	否	CQC	模态组合: CQC/SRSS/ABS/TEN_PERCENT/Dsc/ GRP
components	Sequence	否	()	方向分量: (('幅值名', x 方向余弦, y 方向余弦, z 方向余弦, 旋转分量, 符号), ...)
directDamping	Sequence	否	()	直接阻尼: ((模态号, 阻 尼比, 频率), ...)

(4) 输出参数:

返回值	类型	说明
Step 对象	Step	返回创建的响应谱分析步

3.2.8.6 Model.StaticLinearPerturbationStep()

(1) 作用描述: 创建线性扰动静态步

(2) 示例:

```
mdb.models['Model-1'].StaticLinearPerturbationStep(  
    name='Step-1', previous='Initial')
```

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	—	步名称
previous	String	是	—	前一步名称（首步用'Initial'）
nlgeom	Boolean	否	OFF	是否大变形
description	String	否	"	描述

(4) 输出参数:

返回值	类型	说明
Step 对象	Step	返回创建的线性扰动步

3.2.8.7 SpectrumAmplitude()

(1) 作用描述: 定义响应谱幅值曲线（加速度-频率关系）

(2) 示例:

```
mdb.models['Sign'].SpectrumAmplitude(  
    name='Amp-1', method=DEFINE,  
    specificationUnits=ACCELERATION, gravity=0.0,  
    data=((20.0, 0.1, 0.0), (60.0, 0.2, 0.0),  
          (320.0, 0.5, 0.0), (700.0, 1.0, 0.0),  
          (2500.0, 5.0, 0.0), (1620.0, 10.0, 0.0),  
          (980.0, 50.0, 0.0)))
```

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	—	幅值曲线名称
method	SymbolicConstant	否	DEFINE	DEFINE（自定义） /ENVELOPE（包络）
specificationUnits	SymbolicConstant	否	ACCELERATION	ACCELERATION（加速度） /DISPLACEMENT（位移） /VELOCITY（速度）/ gravity 重力加速度（ m/s^2 ）
gravity	Float	否	9.81	重力加速度（ m/s^2 ）
data	Sequence	否	0	数据点：[(频率 1, 幅值 1, 阻尼 1), (频率 2, 幅值 2, 阻尼 2), ...]

（4）输出参数：

返回值	类型	说明
SpectrumAmplitude 对象	SpectrumAmplitude	返回创建的幅值曲线

3.2.9 BoundaryCondition()

3.2.9.1 Model.BC()

（1）作用描述：创建完全固定边界条件（ $U1=U2=U3=UR1=UR2=UR3=0$ ）

(2) 示例:

```
mdb.models['Model-1'].EncastreBC(name='BC-1',createStepName='Step-1',region=region)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	—	BC 名称
createStepName	String	是	—	创建此 BC 的分析步名称
region	Region	是	—	作用区域(Set/Surface 等)
localCsys	DatumCsys	否	None	局部坐标系 (可选)
Type	String	是	-	PINNED(简支) /ENCASTRE(固支)/XYZSYMM/XYZASYMM

(4) 输出参数:

返回值	类型	说明
bc	BoundaryCondition	边界条件对象

3.2.9.2 Model.DisplacementBC()

(1) 作用描述: 创建位移边界条件

(2) 示例:

```
mdb.models['Model-1'].DisplacementBC(name='BC-2',createStepName='Step-1',region=region,u1=0,u2=UNSET,u3=0,ur1=0,ur2=0,ur3=0)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	-	边界条件名称
createStepName	String	是	-	创建步名称
region	Region	是	-	施加区域
u1,u2,u3	float/UNSET	否	UNSET	X/Y/Z 方向平动约束
ur1,ur2,ur3	float/UNSET	否	UNSET	绕 X/Y/Z 轴转动约束
amplitude	String	否	-	幅值曲线
distributionType	String	否	UNIFORM	分布类型
fieldName	String	否	-	场变量名
localCsys	DatumCsys	否	-	局部坐标系

(4) 输出参数:

返回值	类型	说明
bc	DisplacementBC 对象	创建的边界条件对象

3.2.10 Load()

3.2.10.1 Model.ConcentratedForce()

(1) 作用描述: 创建集中力载荷

(2) 示例:

```
mdb.models['Model-1'].ConcentratedForce(name='Load-1',createStepName='Step-1',region=region,cf1=1.0,cf2=0.0,cf3=0.0,distributionType=UNIFORM)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	-	载荷名称
createStepName	String	是	-	创建步名称
region	Region	是	-	施加区域
cf1,cf2,cf3	float	否	0.0	X/Y/Z 方向 集中力大小
cf1,cf2,cf3	Complex	否	0+0j	X/Y/Z 方向 力 (复数: 实部+虚部 j)
cm1,cm2,cm3	Complex	否	0+0j	X/Y/Z 方向 力矩
amplitude	String	否	UNSET	幅值曲线
distributionType	SymbolicConstant	是	UNIFORM	分布类型
fieldName	String	否	—	场变量名

(4) 输出参数:

返回值	类型	说明
load	ConcentratedForce 对象	创建的载荷对象

3.2.10.2 Model.Pressure()

(1) 作用描述: 创建压力载荷

(2) 示例:

```
mdb.models['Model-1'].Pressure(name='Load-1', createStepName='Step-1',  
region=region, magnitude=50.0)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	—	载荷名称
createStepName	String	是	—	创建步名称
region	Region	是	—	区域对象
magnitude	Float	是	—	压力大小
amplitude	String	否	—	幅值曲线
distributionType	SymbolicConstant	否	UNIFORM	分布类型 (UNIFORM/LINEAR/QUADRATIC/USER)
localCsys	DatumCsys	否	—	局部坐标系

(4) 输出参数:

返回值	类型	说明
Load	Load	载荷对象

3.2.10.3 Model.LineLoad()

(1) 作用描述: 在边上施加线载荷 (梁/壳)

(2) 示例:

```
a.Set(elements=elements1, name='Set-Load')
region = a.sets['Set-Load']
mdb.models['Model-1'].LineLoad(
    name='Load-1', createStepName='Step-1', region=region,
    comp1=20000.0, distributionType=UNIFORM, system=GLOBAL)
```

(2) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	—	载荷名称
createStepName	String	是	—	创建该载荷的分析步名称
region	Region	是	—	作用区域（边集合）
comp1	Float	否	0.0	方向 1 分量（全局坐标系）
comp2	Float	否	0.0	方向 2 分量
comp3	Float	否	0.0	方向 3 分量
magnitude	Float	否	0.0	载荷大小（与 comp 互斥）
distributionType	SymbolicConstant	否	UNIFORM	分布类型：UNIFORM/ANALYTICAL_FIELD
field	String	否	"	场名称 （distributionType≠UNIFORM 时）
system	SymbolicConstant	否	GLOBAL	坐标系： GLOBAL/LOCAL
localCsys	DatumCsys	否	None	局部坐标系 （system=LOCAL 时）
amplitude	String	否	UNSET	幅值曲线名称

(4) 输出参数:

返回值	类型	说明
Load 对象	Load	返回创建的线载荷对象

3.2.10.4 Model.ShellEdgeLoad()

(1) 作用描述：在壳单元的边上施加分布载荷（法向/切向）

(2) 示例：

```
mdb.models['Model-1'].ShellEdgeLoad(  
    name='Load-1', createStepName='Step-1', region=region,  
    magnitude=1.0, traction=NORMAL)
```

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	载荷名称
createStepName	String	是	—	创建该载荷的分析步名称
region	Region	是	—	作用区域（面/边集合）
magnitude	Float	否	0.0	载荷大小
traction	SymbolicConstant	否	NORMAL	载荷方向：NORMAL（法向） /SHEAR/Transverse/ Moment（切向）
distributionType	SymbolicConstant	否	UNIFORM	分布类型
field	String	否	"	场名称
amplitude	String	否	UNSET	幅值曲线名称

(4) 输出参数：

返回值	类型	说明
-----	----	----

返回值	类型	说明
Load 对象	Load	返回创建的边载荷对象

3.2.10.5 Model.Pressure()

(1) 作用描述：在面上施加压力载荷

(2) 示例：

```
mdb.models['Model-1'].Pressure(  
    name='Load-1', createStepName='Step-1', region=region,  
    magnitude=0.1013, distributionType=UNIFORM)
```

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	载荷名称
createStepName	String	是	—	创建该载荷的分析步名称
region	Region	是	—	作用区域（面集合）
magnitude	Float	否	0.0	压力大小（正值=压力）
distributionType	SymbolicConstant	否	UNIFORM	分布类型：UNIFORM /ANALYTICAL_FIELD
field	String	否	"	场名称 （distributionType≠UNIFORM 时）
amplitude	String	否	UNSET	幅值曲线名称

(4) 输出参数：

返回值	类型	说明
-----	----	----

返回值	类型	说明
Load 对象	Load	返回创建的压力载荷对象

3.2.10.6 Model.Moment ()

(1) 作用描述： 施加集中力矩（用于壳/梁单元）

(2) 示例：

```
mdb.models['Model-1'].Moment(
    name='Load-1', createStepName='Step-1', region=region,
    cm1=0, cm2=0, cm3=26.18, distributionType=UNIFORM)
```

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	力矩名称
createStepName	String	是	—	创建该载荷的分析步
region	Region	是	—	作用区域
cm1	Complex	否	0+0j	X 方向力矩
cm2	Complex	否	0+0j	Y 方向力矩
cm3	Complex	否	0+0j	Z 方向力矩
distributionType	SymbolicConstant	否	UNIFORM	分布类型
amplitude	String	否	UNSET	幅值曲线

(4) 输出参数：

返回值	类型	说明
-----	----	----

返回值	类型	说明
Load 对象	Load	返回创建的力矩载荷

3.2.10.7 Model.LoadCase()

(1) 作用描述：在扰动步中定义载荷工况（多工况一次性求解）

(2) 示例：

```
mdb.models['Model-1'].steps['Step-1'].LoadCase(
    name='LoadCase-1', loads=((('Load-1', 1.0), ),
    boundaryConditions=((('BC-1', 1.0), ))
```

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	工况名称
loads	Sequence	否	0	载荷列表： (('载荷名', 系数), ...)
boundaryConditions	Sequence	否	0	边界条件列表： (('BC名', 系数), ...)

(4) 输出参数：

返回值	类型	说明
LoadCase 对象	LoadCase	返回创建的载荷工况

3.2.10.8 DiscreteField()

(1) 作用描述：创建离散场（用于定义单元/节点的局部方向、厚度等）

(2) 示例：

```
mdb.models['Model-1'].DiscreteField(
    name='DiscField-1', description="", location=ELEMENTS,
```


fieldType=ORIENTATION, dataWidth=3, defaultValues=(0.0, 0.0, 0.0),
data=((" 3, (4, 5, 2, 3), (0.25, 0.0, 0.0, 0.25, 0.0, 0.0, 0.0, 0.0, -0.25, 0.0, 0.0)),))

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	—	场名称
description	String	否	"	描述
location	SymbolicConstant	是	—	作用位置: NODES/ELEMENTS/FACES/CELLS
fieldType	SymbolicConstant	是	—	场类型: ORIENTATION/THICKNESS/OFFSET/TRACTION 等
dataWidth	Int	否	1	数据宽度(每个位置的数据个数)
defaultValues	Sequence	否	(0.0,)	默认值序列
data	Sequence	否	()	数据元组: (区域标签, 数据宽度, 区域 ID 序列, 数据值序列)

(4) 输出参数:

返回值	类型	说明
Field 对象	Field	返回创建的离散场对象

3.2.10.9 MappedField()

(1) 作用描述：创建映射场（将离散数据点插值为连续场，用于非均匀载荷）

(2) 示例：

```
mdb.models['Model'].MappedField(  
    name='AnalyticalField-1', description="",  
    regionType=POINT, partLevelData=False, localCsys=None,  
    pointDataFormat=XYZ, fieldDataType=SCALAR,  
    xyzPointData=((3.67, -25, 8.401, -91654.2), (16.547, -25, 10.152, -76446.4),  
        ...200+个点...))
```

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	场名称
description	String	否	"	描述
regionType	SymbolicConstant	否	CELL	POINT（点）/CELL（单元）
partLevelData	Boolean	否	False	False=模型级, True=部件级
localCsys	DatumCsys	否	None	局部坐标系
pointDataFormat	SymbolicConstant	否	XYZ	XYZ（坐标+值）/CYLINDRICAL/SPHERICAL
fieldDataType	SymbolicConstant	否	SCALAR	SCALAR（标量）/VECTOR（矢量）/TENSOR（张量）
xyzPointData	Sequence	否	0	数据点： ((x1,y1,z1,v1), (x2,y2,z2,v2), ...)
neighborhoodSea	Float	否	1e+0	邻域搜索容差

参数	类型	必需	默认值	说明
rchTol			6	
isZeroPressureHeight	Boolean	否	False	是否零压高度（流体）

（4）输出参数：

返回值	类型	说明
Field 对象	Field	返回创建的映射场

3.2.10.10

3.2.10.11

(1) 作用描述:

(2) 示例:

(3) 输入参数:

(4) 输出参数:

3.2.10.12

(1) 作用描述:

(2) 示例:

(3) 输入参数:

(4) 输出参数:

3.2.10.13

(1) 作用描述:

(2) 示例:

(3) 输入参数:

(4) 输出参数:

3.2.10.14

(1) 作用描述:

(2) 示例:

(3) 输入参数:

(4) 输出参数:

3.2.10.15

(1) 作用描述:

(2) 示例:

(3) 输入参数:

(4) 输出参数:

3.2.10.16

(1) 作用描述:

(2) 示例:

(3) 输入参数:

(4) 输出参数:

3.2.10.17

(1) 作用描述:

(2) 示例:

(3) 输入参数:

(4) 输出参数:

3.2.11 Interaction()

3.2.11.1 Model.Coupling()

(1) 作用描述：创建耦合约束（将控制节点与从属面关联，实现 MPC/Coupling）

(2) 示例：

```
mdb.models['Model-1'].Coupling(  
    name='Constraint-1', controlPoint=region1, surface=region2,  
    couplingType=KINEMATIC, u1=ON, u2=ON, u3=ON, ur1=ON, ur2=ON, ur3=ON)
```

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	约束名称
control Point	Region	是	—	控制节点区域（主面）
surface	Region	是	—	从属面区域
couplingType	Symbolic Constant	否	KINEMATIC	耦合类型： KINEMATIC/DISTRIBUTING/TIE/ CONTINUUM_DISTRIBUTING
u1	Boolean	否	ON	平动 1 方向：ON=约束, OFF=自由
u2	Boolean	否	ON	平动 2 方向
u3	Boolean	否	ON	平动 3 方向
ur1	Boolean	否	OFF	转动 1 方向
ur2	Boolean	否	OFF	转动 2 方向
ur3	Boolean	否	OFF	转动 3 方向

(4) 输出参数：

返回值	类型	说明
Constraint 对象	Constraint	返回创建的耦合约束对象

3.2.11.2 Model.MFluidProp()

(1) 作用描述：定义流体属性（用于声学/流体耦合分析）

(2) 示例：

```

mdb.models['Model'].MFluidProp(
    name='IntProp-1', height=10.0, density=1025.0, freeSurface=True)

```

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	流体属性名称
height	Float	否	0.0	流体高度（m）
density	Float	否	1000.0	流体密度（kg/m ³ ）
freeSurface	Boolean	否	False	是否自由液面

(4) 输出参数：

返回值	类型	说明
MFluidProp 对象	MFluidProp	返回创建的流体属性

3.2.11.3 Model.MFluid()

(1) 作用描述：定义流体-结构相互作用

(2) 示例：

```

mdb.models['Model'].MFluid(
    name='Interaction-1', createStepName='Step-1',

```

surface=region, interactionProperty='IntProp-1')

(3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	—	相互作用名称
createStepName	String	是	—	创建该相互作用的分析步
surface	Region	是	—	流体作用面（壳/膜单元表面）
interactionProperty	String	是	—	流体属性名称（MFluidProp）

(4) 输出参数:

返回值	类型	说明
Interaction 对象	Interaction	返回创建的流体相互作用

3.2.12 Mesh()

3.2.12.1 Part.createNode()

(1) 作用描述: 在部件中创建节点

(2) 示例:

```
p = mdb.models['Model-1'].parts['Part-1']
```

```
p.createNode(x=0, y=0, z=0)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
x	Float	否	0.0	X 坐标

参数	类型	必需	默认值	说明
y	Float	否	0.0	Y 坐标
z	Float	否	0.0	Z 坐标

(4) 输出参数:

返回值	类型	说明
Node 对象	Node	返回创建的节点对象

3.2.12.2 Part.Element()

(1) 作用描述: 创建单元

(2) 示例:

```
p = mdb.models['Model-1'].parts['Part-1']
```

```
n = p.nodes
```

```
p.Element(nodes=(n[0], n[1]), elemShape=BEAM, intersectNodes=True)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
nodes	Sequence	是	—	节点序列（按单元连接顺序）
elemShape	SymbolicConstant	是	—	单元形状: BEAM/TRUSS/Quad 4 /Tri3 等
mergeNodes	Boolean	否	False	是否合并重合节点
mergeElements	Boolean	否	False	是否合并重合单元

参数	类型	必需	默认值	说明
intersectNodes	Boolean	否	False	是否在节点处打断

(4) 输出参数:

返回值	类型	说明
Element 对象	Element	返回创建的单元对象

3.2.12.3 Part.extrude()

(1) 作用描述: 沿指定路径拉伸区域生成新单元

(2) 示例:

```

region = regionToolset.Region(nodes=nodes1)
p.extrude(region=region,    type=POINTTOLINE,    distance=(40.0,    0.0,    0.0),
copyNumbers=20)

```

(3) 输入参数:

参数	类型	必需	默认值	说明
region	Region	是	—	要拉伸的区域对象
type	SymbolicConstant	否	TRANSLATION	拉伸类型: LINETOSHELL/POINTTOLINE/SHELLTOSOLID
distance	Float/Sequence	否	—	拉伸距离 (标量或向量)
copyNumbers	Int	否	1	复制份数

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。新单元被添加到部件

3.2.12.4 Part.breakBeam()

(1) 作用描述：将梁单元打断为多段

(2) 示例：

```
e1 = p.elements
```

```
elements1 = e1.getSequenceFromMask(mask=['#1'], )
```

```
p.breakBeam(elements=elements1, type=QUANTITY, number=2)
```

(3) 输入参数：

参数	类型	必需	默认值	说明
elements	Sequence	是	—	要打断的梁单元序列
type	SymbolicConstant	否	QUANTITY	打断方式： QUANTITY（按数量）/RATIO（按比例）/LENGTH（按长度）
number	Int	否	2	打断份数 （type=QUANTITY 时）
ratio	Float	否	0.5	比例 （type=RATIO 时，0<ratio<1）
length	Float	否	—	每段长度 （type=LENGTH 时）
DISTANCE	Float	否	—	按距离打断起始

参数	类型	必需	默认值	说明
				端头、末尾端头
ByNodes	Int	否	—	按节点打断
Byprojection	Int	否	—	垂直投影打断梁

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。原单元被打断为多个新单元

3.2.12.5 Part.createArc()

(1) 作用描述: 创建圆弧边 (连接三个节点, 中间节点为圆弧上的点)

(2) 示例:

`p.createArc(nodes=(node1[1], node1[0], node1[5]), segments=8, isArc=True)`

(3) 输入参数:

参数	类型	必需	默认值	说明
nodes	Sequence	是	—	三个节点序列: (起点, 弧上点, 终点)
segments	Int	否	3	圆弧分段数
isArc	Boolean	否	True	True=创建圆弧, False=创建整圆

(4) 输出参数:

返回值	类型	说明
-----	----	----

返回值	类型	说明
Edge 对象	Edge	返回创建的圆弧边对象

3.2.12.6 Part.fillet()

(1) 作用描述：在两个区域之间创建圆角（倒角）

(2) 示例：

```
region = regionToolset.Region(face1Elements=face1Elements)
```

```
region1 = regionToolset.Region(face1Elements=face1Elements)
```

```
p.fillet(region=region, region1=region1, radius=50.0, segments=4)
```

(3) 输入参数：

参数	类型	必需	默认值	说明
region	Region	是	—	第一个区域（面/边集合）
region1	Region	是	—	第二个区域（面/边集合）
radius	Float	否	—	圆角半径
segments	Int	否	4	圆角分段数

(4) 输出参数：

返回值	类型	说明
无	None	无返回值。圆角特征被添加到部件

3.2.12.7 Part.meshRebuild()

(1) 作用描述：重建指定单元的网格

(2) 示例：

```
p.meshRebuild(elements=elements1,meshSize=50.0,keepBoundary=False)
```

(3) 输入参数：

参数	类型	必需	默认值	说明
elements	sequence	是	—	要重划的单元序列
meshSize	Float	是	—	目标网格尺寸
keepBoundary	Boolean	否	False	是否保持边界节点不变
maxSize	Float	否	None	最大网格尺寸（可选）
minSizeFactor	Float	否	None	最小尺寸因子（可选）

（4）输出参数：

返回值	类型	说明
无	None	无返回值。指定区域网格被重建

3.2.12.8 Part.createShellByPolygon()

（1）作用描述：通过闭合多边形区域生成壳单元

（2）示例：

```
region = regionToolset.Region(face1Elements=face1Elements)
```

```
p.createShellByPolygon(region=region, meshSize=0.0, keepBoundary=True,  
deleteBeam=True)
```

（3）输入参数：

参数	类型	必需	默认值	说明
region	Region	是	—	边界区域（边集合）
meshSize	Float	否	0.0	网格尺寸（0=自动）
keepBoundary	Boolean	否	True	是否保留边界单元
deleteBeam	Boolean	否	False	是否删除原始梁单元

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。壳单元被添加到部件

3.2.12.9 Part.mergeNodes()

(1) 作用描述: 合并距离容差内的重合节点

(2) 示例:

`p.mergeNodes(state=0, tolerance=0.00499999988824129, keepHighLabel=False)`

(3) 输入参数:

参数	类型	必需	默认值	说明
state	Int	否	0	合并状态: 0=全局合并, 1=按选择的节点区域进行合并
tolerance	Float	否	—	容差距离
keepHighLabel	Boolean	否	True	True=保留高标号, False=保留低标号

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。重合节点被合并

3.2.12.10 Part.breakShell()

(1) 作用描述: 将壳单元打断为多段

(2) 示例:

`region = regionToolset.Region(elements=elements1)`

`p.breakShell(region=region, type=TWOQUAD, onSelectedEdge=False)`

(3) 输入参数:

参数	类型	必需	默认值	说明
region	Region	是	—	要打断的壳单元区域
type	SymbolicConstant	否	TWOQUAD	打断方式: TWOQUAD/ FOURQUAD / THREEQUAD/ FOURQUADATEDGE/ QUADTRIATLONGER EDGE/ TWOTRI
onSelectedEdge	Boolean	否	False	是否在选中边上打断
OnLongestEdge	Boolean	否	True	是否在最长边打断

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。原壳单元被打断为多个新单元

3.2.12.11 Part.mirrorElement()

(1) 作用描述: 镜像单元到对称面另一侧

(2) 示例:

```
p.mirrorElement(elements=elements1, isCoordinatePlane=True,  
principalPlane=YZPLANE, offset=300.0, copyProperties=True)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
elements	Sequence	是	—	要镜像的单元序列

参数	类型	必需	默认值	说明
isCoordinate Plane	Boolean	否	False	True=坐标平面, False=指定平面
principalPlane	SymbolicConstant	否	XYPLANE	主平面: XYPLANE/YZPLANE/ZXPLANE
offset	Float	否	0.0	镜像偏移距离
copyProperties	Boolean	否	False	是否复制属性

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。镜像单元被添加到部件

3.2.12.12 Part.translateElement()

(1) 作用描述: 平移复制单元 (生成新单元)

(2) 示例:

```
p.translateElement(
    elements=elements1, copy=True, distance=(180., 0., 0.),
    numberOfTimes=2, copyProperties=True)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
elements	Sequence	是	—	要平移的单元序列
copy	Boolean	否	False	True=复制生成新单元, False=仅平移原单元

参数	类型	必需	默认值	说明
distance	Sequence	否	(0.0, 0.0, 0.0)	平移向量 (dx, dy, dz)
numberOfTimes	Int	否	1	复制份数 (copy=True 时)
copyProperties	Boolean	否	False	是否复制属性 (截面、材料等)
moveNodes	Boolean	否	True	是否同时平移节点

(4) 输出参数:

返回值	类型	说明
无	None	无返回值。新单元被添加到部件

3.2.12.13 Part.openingMesh()

(1) 作用描述: 网格开孔

(2) 示例:

```
p.openingMesh(nodes=(n[220], n[224], n[265]), type=CIRCLEHOLE,
               radius=200.0, meshSize=50.0, edgeLayers=2, meshRange=3.0)
```

(3) 输入参数:

参数	类型	必需	默认值	说明
nodes	Sequence	是	—	定义开口的节点列表 (3 个点确定圆)
type	SymbolicConstant	否	CIRCLEHOLE	CIRCLE(圆孔)/Manhole(人孔)/Rectangle

参数	类型	必需	默认值	说明
				(长方形) /Rounded Rectangle
radius	Float	否	—	圆孔半径
meshSize	Float	否	—	开口边缘网格 尺寸
edgeLayers	Int	否	0	边缘网格层数
meshRange	Float	否	0.0	网格影响范围 (从边缘向外 扩展)

(4) 输出参数:

返回值	类型	说明
无	None	在部件上创建开口

3.2.12.14 SetPartElementTypes

(1) 作用描述: 设置部件默认单元类型控制

(2) 示例:

```
mdb.models['Model-1'].parts['Part-1'].SetPartElementTypes(isInclude=True,
    isReduced=False, stressCheckedId=True, standardCheckedId=True,
    solidisReducedIntegr=True, isIncompatible=False, isEulerianCheckedId=False)
```

(3) 输入参数:

参数	值	含义
isInclude	True	包含单元类型 (不是排除)

参数	值	含义
isReduced	False	壳/梁单元用完全积分
stressCheckedId	True	开启应力检查
standardCheckedId	True	开启标准单元检查
solidisReducedIntegr	True	实体单元用缩减积分
isIncompatible	False	不用非协调模式
isEulerianCheckedId	False	不检查欧拉单元

(4) 输出参数:

输出	类型	说明
无	None	该方法无返回值，仅修改模型属性

3.2.13 Job()

3.2.13.1 Job.Create()

- (1) 作用描述: 创建求解作业
- (2) 示例: `mdb.Job(name='Test1',model='Model-1')`
- (3) 输入参数:

参数	类型	必需	默认值	说明
name	String	是	—	作业名称
model	Model	是	—	关联的模型
type	Symbol	否	—	Input File

(4) 输出参数:

返回值	类型	说明
job	Job	作业对象，后续可调用.submit()、.waitForCompletion()等

3.2.13.2 Job.submit()

(1) 作用描述：提交作业进行求解

(2) 示例：mdb.jobs['Test1'].submit(h5=ON,Compatibility=OFF)

(3) 输入参数：

参数	类型	必需	默认值	说明
h5	Boolean	否	ON	ON=生成.h5
Compatibility	Symbol	否	OFF	ON/OFF，是否兼容旧版本

(4) 输出参数：

返回值	类型	说明
无	None	无返回值。作业被提交到队列

3.3 Odb 内置方法

3.3.1 importHDF5FileNewOdb()

(1) 作用描述：导入.h5 格式 ODB 文件

(2) 示例：o=session.importHDF5FileNewOdb(name='D:/Temp/Test1.h5')

(3) 输入参数：

参数	类型	必需	默认值	说明
name	String	是	—	.h5 文件路径

参数	类型	必需	默认值	说明
odb	Odb	否	None	可指定已有 Odb 对象（可选）

（4）输出参数：

返回值	类型	说明
odb	Odb	ODB 对象，后续可调用 <code>.steps</code> 、 <code>.frames</code> 等

3.3.2 Model.exportOdbToVtk()

（1）作用描述：导出为 VTK 格式

（2）示例：

```
mdb.models['Model-1'].exportOdbToVtk(
'D:/Temp2/Test1','D:/Temp2/Test1.h5',4,'Model-1')
```

（3）输入参数：

参数	类型	必需	默认值	说明
vtkFileName	String	是	—	输出 VTK 目录名（不含扩展名）
odbFileName	String	是	—	源 ODB/H5 文件路径
numFrames	Int	是	—	导出帧数
modelName	String	是	—	模型名称

（4）输出参数：

返回值	类型	说明
无	None	无返回值。VTK 文件被生成